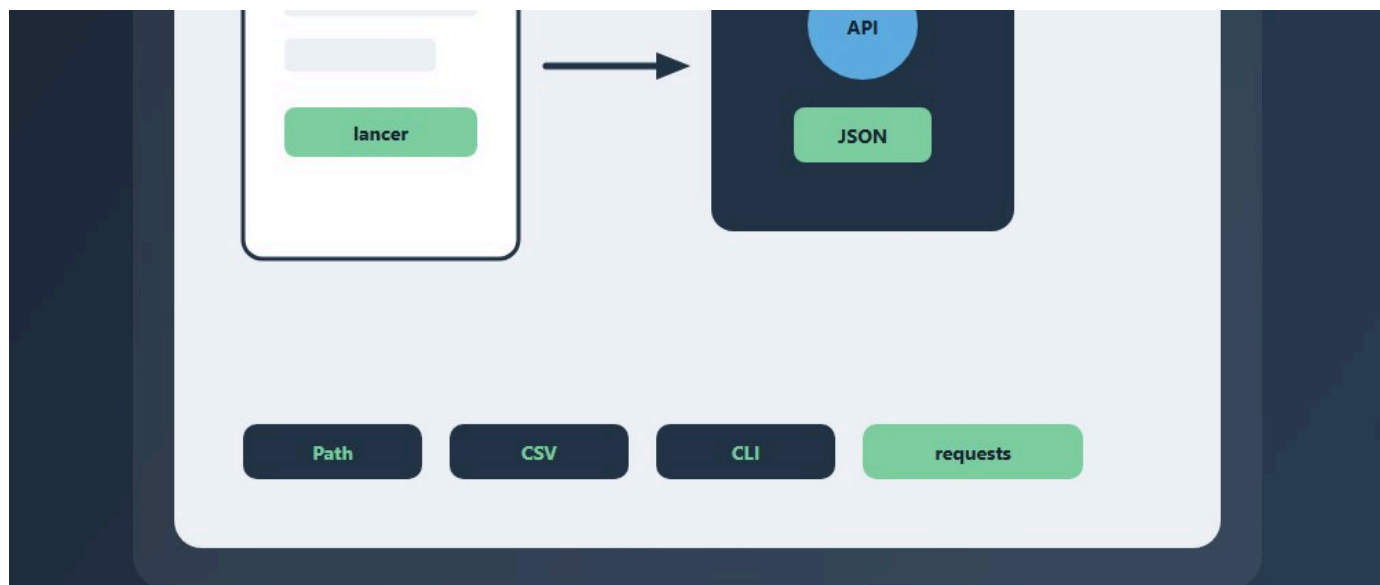




FORMATION INFORMATIQUE - PRATIQUE

Python - Pratique

CSV, CLI, venv, API et bons reflexes : la
suite utile apres les bases Python.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. Chapitre 1 - Rappel rapide et carte du livre

- [Ce que ce n'est pas](#)
- [Ce que tu gardes en tete](#)
- [La carte du livre](#)
- [Ce dont tu as besoin](#)
- [Comment lire ce livre](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

2. Chapitre 2 - pathlib : des chemins sans douleur

- [Creer un Path](#)
- [Existe ? Fichier ? Dossier ?](#)
- [Lire et ecrire du texte](#)
- [Lister un dossier](#)
- [Chemin absolu vs relatif](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

3. Chapitre 3 - CSV : lire et ecrire un tableau

- [Un exemple concret : notes](#)
- [Lire avec csv.DictReader](#)
- [Ecrire avec csv.DictWriter](#)
- [Separeteur point-virgule](#)
- [Calculer une moyenne](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

4. Chapitre 4 - argparse : un script avec des arguments

- [Premier exemple](#)
- [Types et drapeaux](#)
- [Relier au CSV](#)
- [Arguments positionnels](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

5. Chapitre 5 - venv et pip : isoler les paquets

- [Creer un venv](#)
- [L'activer](#)
- [Installer un paquet](#)
- [Verifier](#)
- [.gitignore](#)
- [Mettre a jour pip \(souvent utile\)](#)
- [Plusieurs projets, plusieurs venv](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

6. Chapitre 6 - requests GET : appeler une API JSON

- [Installer \(rappel\)](#)
- [Premier appel](#)
- [Meteo : exemple concret](#)
- [Afficher un resume](#)
- [Headers \(rapide\)](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : API = contrat](#)

7. Chapitre 7 - Erreurs HTTP : status, timeout, try/except

- [status code et raise for status](#)
- [Timeout](#)
- [JSON pourri](#)
- [Pattern solide pour un mini outil](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : deux canaux](#)

8. Chapitre 8 - Variables d'environnement

- [Lire une variable](#)
- [Definir la variable avant de lancer](#)
- [Le fichier .env](#)
- [Charger .env a la main \(mini version\)](#)
- [Config non secrete](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

9. Chapitre 9 - logging basique

- Premier logger
- Niveaux utiles
- Logger nomme
- Ecrire dans un fichier
- Avec requests / CSV
- print ou logging ?
- Exception + log
- Petite histoire
- Erreur classique
- En vrai
- A toi

10. Chapitre 10 - datetime pratique

- Ce que ce n'est pas
- Maintenant
- Formater pour afficher
- Parser une chaine
- Ajouter / retirer des jours
- Filtrer des lignes "du mois"
- Timestamp de log
- Petite histoire
- Erreur classique
- En vrai
- A toi

11. Chapitre 11 - Regex leger avec re

- search : y a-t-il un motif ?
- match vs search vs fullmatch
- Groupes
- findall
- Cas utiles du quotidien
- Quand ne pas regex
- Petite histoire
- Erreur classique
- Zoom : lire un log avec regex
- En vrai
- A toi

12. Chapitre 12 - Mini-projet : CLI resume CSV ou API

- Parcours
- Squelette argparse
- Mode notes (coeur)
- Mode meteo (coeur)
- Qualite minimale
- Organisation minimale du fichier
- Petite histoire

- [Variante](#)
- [Criteres "c'est fini"](#)
- [A toi](#)

13. [Chapitre 13 - A retenir \(carte mentale\)](#)

- [Ce que ce n'est pas](#)
- [Les idees solides](#)
- [La boucle DanielCraft](#)
- [Ce que tu peux oublier](#)
- [Petite checklist de poche](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

14. [Chapitre 14 - Atelier : CSV des notes](#)

- [Exercice 1 - Preparer les donnees \(10 min\)](#)
- [Exercice 2 - Lire et calculer \(15 min\)](#)
- [Exercice 3 - Ecrire le resume \(10 min\)](#)
- [Exercice 4 - Gerer l'echec \(5 min\)](#)
- [Idee de structure interne](#)
- [Petite histoire](#)
- [Livrable](#)
- [Criteres de reussite](#)
- [Bonus](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

15. [Chapitre 15 - Atelier : CLI utile](#)

- [Exercice 1 - Arguments de base \(10 min\)](#)
- [Exercice 2 - Options avancees \(10 min\)](#)
- [Exercice 3 - Cas d'echec \(10 min\)](#)
- [Exercice 4 - Montrer a quelqu'un \(5 min\)](#)
- [Exemple d'appel](#)
- [Petite histoire](#)
- [Livrable](#)
- [Criteres de reussite](#)
- [Bonus](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

16. [Chapitre 16 - Atelier : appel API meteo](#)

- [Exercice 1 - Environnement \(5 min\)](#)
- [Exercice 2 - Appel de base \(10 min\)](#)
- [Exercice 3 - Extraction et affichage \(10 min\)](#)

- [Exercice 4 - Echecs propres \(10 min\)](#)
- [Exercice 5 - Variable d'environnement \(5 min\)](#)
- [Squelette d'extraction](#)
- [Petite histoire](#)
- [Livrable](#)
- [Criteres de reussite](#)
- [Bonus](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

17. [Chapitre 17 - Tests simples](#)

- [Ce que ce n'est pas](#)
- [assert : la base](#)
- [Separer le coeur du CLI](#)
- [L'idee pytest](#)
- [Quoi tester en priorite ?](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [Fichier checks.py minimal](#)
- [En vrai](#)
- [A toi](#)

18. [Chapitre 18 - Organiser un petit projet](#)

- [Ce que ce n'est pas](#)
- [Arborescence type](#)
- [requirements.txt](#)
- [README court](#)
- [.gitignore utile](#)
- [Ou mettre le point d'entree ?](#)
- [Variante : tout a plat au debut](#)
- [Commandes types dans le README](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

19. [Chapitre 19 - Bonnes pratiques](#)

- [Fonctions courtes, noms clairs](#)
- [Un seul role pour le point d'entree](#)
- [Echecs previsibles](#)
- [UTF-8 et chemins](#)
- [Dependances sous controle](#)
- [Secrets](#)
- [Peu de magie](#)
- [Documenter le "pourquoi" utile](#)
- [Versionne tes donnees d'exemple](#)

- [Petit, mais relancable](#)
- [Eviter la perfection prematuree](#)
- [Code retour et scripts en chaine](#)
- [Petite checklist avant de partager](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

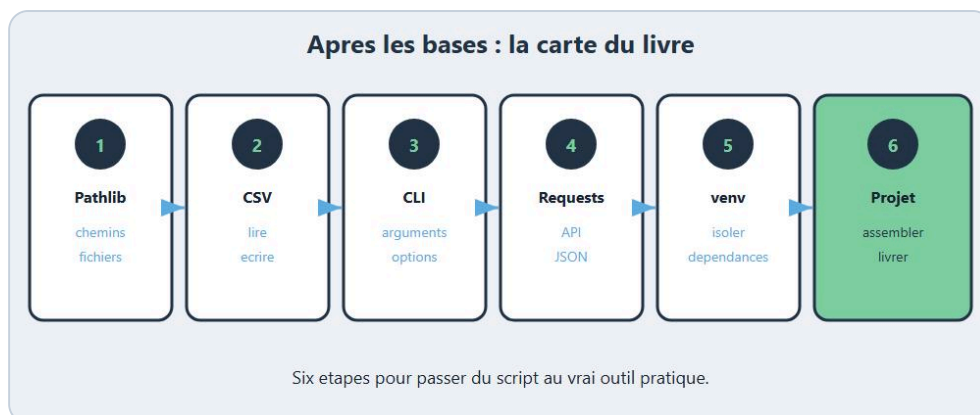
20. [Quiz final](#)

- [Questions](#)
- [Corriges](#)
- [Petite histoire](#)
- [Questions bonus \(auto-eval\)](#)
- [En vrai](#)
- [A toi](#)
- [Pour aller plus loin sans te perdre](#)

21. [Chapitre 21 - Bravo](#)

- [Ce que ce n'est pas](#)
- [Ce que tu sais faire maintenant](#)
- [La suite](#)
- [Un conseil pour la route](#)
- [Un dernier rappel](#)
- [Engagement de 30 jours](#)
- [Petite histoire](#)
- [En vrai](#)
- [A toi](#)

Chapitre 1 - Rappel rapide et carte du livre



Après les bases : chemins, CSV, CLI, API, projet propre.

Tu as déjà fait les bases. Variables, conditions, boucles, fonctions, listes, dictionnaires, fichiers texte, JSON, modules, exceptions, et même un peu de classes. Si un détail est flou, ce n'est pas grave : on ne va pas tout refaire. Ce livre part du moment où Python devient un outil du quotidien, pas un exercice de cours. Tu n'écris plus pour "passer le TD". Tu écris pour gagner du temps, chaque semaine, sur des tâches réelles.

Chez DanielCraft, on aime bien une image simple. Les bases, c'était apprendre à conduire sur un parking. La **pratique**, c'est prendre la route : ouvrir un fichier **CSV** de notes, lancer un script avec des arguments, appeler une **API** météo, isoler tes paquets dans un **venv**, et laisser des traces utiles quand ça casse. Lea, freelance web, veut automatiser ses exports clients. Max, artisan, veut résumer ses factures sans Excel à minuit. Sam, enseignant, veut traiter les notes de ses élèves en une commande. Trois métiers, même logique : Python comme outil, pas comme devoir.

Trois fils rouges reviennent tout le livre. Un fichier CSV de notes (élève, matière, note) que Sam utilise en classe. Une API météo qui renvoie du JSON que Max consulte avant un chantier extérieur. Un script **CLI** que Lea lance depuis le terminal avec des options, sans retoucher le code à chaque client. Ces exemples ne sont pas décoratifs. Ils te montrent le même geste sous plusieurs angles : lire des données, les résumer, gérer l'échec, rendre le script utilisable par quelqu'un d'autre - toi demain matin compte.

Ce que ce n'est pas

Ce n'est pas un cours complet depuis zéro. On ne revient pas sur `print`, `if`, `for`, ni sur "c'est quoi un dictionnaire". Si tu bloques sur ces mots, relis le premier livre Python du parcours. Ce n'est pas non plus un livre web : pas de Django, Flask, FastAPI ici. On reste en Python "outil", clair, dans le terminal et les fichiers. Ce n'est pas un manuel de data science non plus. Pas de pandas obligatoire. Juste la bibliothèque standard plus `requests` et éventuellement `pytest`.

Ce n'est pas non plus "devenir expert en une semaine". Un script qui lit un tableau et affiche une moyenne, c'est déjà du temps gagné. Un script qui affiche la météo du matin, c'est déjà un outil perso. Un CLI avec `-h` lisible, c'est déjà "pro" pour un petit projet. On monte cran par cran, sans te noyer.

★ A RETENIR

Note des maintenant tes trois fils rouges perso : un tableau à lire, une info à récupérer dehors, une commande que tu voudrais taper sans ouvrir l'éditeur.

Ce que tu gardes en tete

Tu sais ecrire une fonction. Tu sais boucler sur une liste. Tu sais ouvrir un fichier texte et lire du JSON. Tu as vu `try/except`. C'est assez pour avancer. On monte d'un cran avec des outils standards : `pathlib` pour les chemins, `csv` pour les tableaux, `argparse` pour la ligne de commande, `venv` et `pip` pour les paquets, `requests` pour le HTTP, `logging` pour les traces, `datetime` pour les dates, `re` pour quelques motifs texte.

La carte du livre

Les premiers chapitres posent les briques une par une. Chemins, CSV, CLI, environnement virtuel, appels API, erreurs reseau, secrets, logs, dates, regex. Puis un mini-projet qui assemble tout ca. Trois ateliers concrets (CSV, CLI, API). Ensuite tests simples, organisation de projet, bonnes pratiques. Le quiz verifie. Le dernier chapitre, c'est pour souffler et regarder la route parcourue.

Tu n'as pas besoin d'un framework web pour etre utile des demain. Ce livre te donne la boite a outils du quotidien : fichiers, terminal, reseau, hygiene. Le reste (web, bases de donnees, packaging) vient apres, sur ce socle.

Ce dont tu as besoin

Python 3 installe. Un editeur (VS Code, Cursor, ou autre). Un terminal. Pour la partie API : une connexion internet et un venv. Rien de plus pour suivre. Les paquets hors standard seront installes au fur et a mesure (`requests` , plus tard eventuellement `pytest`).

Comment lire ce livre

Lis dans l'ordre au debut. Les chapitres s'enchainent logiquement. Les ateliers sont la pour faire, pas seulement lire. A chaque fin de chapitre, il y a un "A toi". Fais-le. Cinq minutes valent mieux qu'une lecture passive. Tu peux revenir ensuite a un chapitre precis (CSV, argparse, requests) comme a une fiche quand un cas reel te bloque.

Petite histoire

Sam avait un tableur de notes ouvert depuis septembre. Chaque fin de trimestre, meme galere : copier-coller, moyennes a la main, une formule cassee. Lea lui a montre un script de quarante lignes qui lit un CSV et sort les moyennes. Sam n'est pas devenu developpeur. Il a juste gagne deux heures par session. Max, lui, checkait la meteo sur son telephone avant chaque toiture. Il a ecrit un petit script terminal avec sa ville en argument. Moins de clics, meme info, plus rapide le matin dehors.

Lea, le meme mois, a transforme trois exports clients "a la main" en une commande. Elle a gagne une soiree. Pas de magie. Juste Python pratique, le genre d'outil qu'on garde sous le coude chez DanielCraft.

Erreur classique

Croire que "je connais les bases" egal "je sais automatiser mon quotidien". Les bases sont le moteur. La pratique, c'est la boite a outils : CSV, CLI, HTTP, logs. Sans ca, tu reecris tout a la main a chaque fois. Autre piege : vouloir tout apprendre d'un coup sans faire les ateliers. Ce livre se gagne en tapant du code, pas en lisant passivement.

⚠ ATTENTION

Si tu sautes les "A toi", tu vas "comprendre" et ne plus savoir faire dans trois semaines. Cinq minutes actives battent quarante pages passives.

En vrai

Ouvre un vieux script Python que tu as déjà écrit. Note ce qui manque pour le rendre utile : lire un tableau ? accepter un argument ? appeler une URL ? isoler les paquets ? Ce livre répond à ça, chapitre par chapitre.

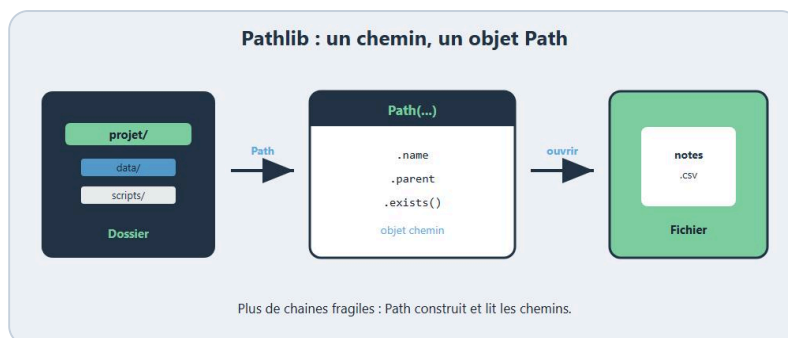
A toi

Écris en trois phrases ce que tu veux construire à la fin. Pas un réseau social. Quelque chose de petit : "résumer mes notes depuis un CSV", "afficher la météo de ma ville", "un script CLI qui dit bonjour avec mon nom". Garde ce but. On y reviendra au mini-projet. Si tu hésites entre trois idées, choisis celle que tu utiliserais vraiment la semaine prochaine. Chez DanielCraft, un usage réel bat une intention ambitieuse.

♦ ASTUCE

Les bases, c'est le moteur. Ce livre, c'est la boîte à outils du quotidien : fichiers, CLI, HTTP, hygiène. Garde cette phrase sous le coude.

Chapitre 2 - pathlib : des chemins sans douleur



Path : des chemins clairs, sans galere de slash.

En Python, tu touches souvent a des fichiers. Un CSV de notes, un dossier `data`, un fichier de config. Le piege classique, c'est de coller des chemins a la main avec des chaines et des slash... jusqu'au jour ou ca casse sur Windows, ou sur un autre dossier, ou quand tu lances le script depuis un endroit different. Tu perds une heure a chercher un fichier qui "existait hier".

`pathlib` est la solution moderne : tu travailles avec des objets **Path**, pas avec des string brutes. Un chemin, c'est une adresse. Avec `pathlib`, tu manipules l'adresse comme un objet : tu l'assembles, tu verifies si elle existe, tu lis ou tu ecris. L'operateur `/` remplace le collage manuel de slash. Python s'occupe du detail selon le systeme. Tu n'as plus a te demander si c'est `\` ou `/`. Tu penses "dossier puis fichier", pas "string fragile".

Chez DanielCraft, on resume comme ca : un chemin, c'est un objet, pas un texte magique. Lea ancre ses scripts sur `Path(__file__).parent` pour que les exports clients marchent quel que soit le dossier courant. Max range ses CSV de factures dans `data/` sans se battre avec les backslash Windows. Sam garde ses notes dans un dossier a cote du script, pas "quelque part sur le bureau".

♦ ASTUCE

Des que ton fichier doit vivre "a cote" du script, ancre avec `Path(__file__).resolve().parent`. Tu evites la moitie des "fichier introuvable".

Creer un Path

```
from pathlib import Path

dossier = Path("data")
fichier = Path("data") / "notes.csv"
```

L'operateur `/` assemble les morceaux. Sur Windows comme sur Linux, ca marche. Tu peux aussi partir du dossier du script :

```
ici = Path(__file__).resolve().parent
notes = ici / "data" / "notes.csv"
```

`__file__` est le chemin du fichier `.py` en cours. `.parent` remonte d'un cran. C'est utile quand tu lances le script depuis un autre repertoire : tu ne depend plus du "dossier courant" mysterieux.

Existe ? Fichier ? Dossier ?

```
p = Path("data/notes.csv")
print(p.exists())
print(p.is_file())
print(p.parent.is_dir())
```

Avant d'ouvrir, tu peux vérifier. Ce n'est pas obligatoire à chaque ligne, mais c'est rassurant au début. Si le dossier n'existe pas, tu peux le créer :

```
Path("data").mkdir(parents=True, exist_ok=True)
```

`parents=True` crée aussi les dossiers parents manquants. `exist_ok=True` évite une erreur si le dossier est déjà là.

Lire et écrire du texte

Pour un petit fichier texte (ou un JSON en string), `pathlib` a des raccourcis :

```
chemin = Path("data/bonjour.txt")
chemin.write_text("Salut\n", encoding="utf-8")
contenu = chemin.read_text(encoding="utf-8")
print(contenu)
```

Toujours préciser `encoding="utf-8"` quand tu manipules du français. Sinon, selon la machine, les accents peuvent devenir bizarres. Pour du binaire (images, etc.), il existe `read_bytes` / `write_bytes`. On reste sur du texte ici.

Lister un dossier

```
dossier = Path("data")
for element in dossier.iterdir():
    print(element.name, element.is_file())
```

Ou filtrer avec un motif :

```
for csv in Path("data").glob("*.csv"):
    print(csv)
```

`rglob` fait la même chose en récursif (sous-dossiers inclus). Pratique pour trouver tous les `.csv` d'un projet.

Chemin absolu vs relatif

Un chemin **relatif** dépend du dossier où tu lances Python. Un chemin **absolu** pointe toujours au même endroit.

```
p = Path("data/notes.csv")
print(p.resolve())
```

`.resolve()` donne la forme absolue, avec les `..` ranges. Quand tu debugues "fichier introuvable", affiche `resolve()` : souvent tu regardais au mauvais endroit.

Petite histoire

Max avait un script qui lisait `notes.csv` en dur. Ca marchait quand il lançait depuis son dossier projet. Un matin, il l'a lancé depuis `C:\Users\Max` et Python a dit "fichier introuvable". Il a cru que Python était cassé. En réalité, le chemin relatif pointait ailleurs. Avec `Path(__file__).parent / "data" / "notes.csv"`, le script retrouve toujours son fichier, peu importe d'où tu l'appelles.

Lea a le même réflexe sur tous ses outils clients. Sam le montre en cours : "change de dossier, relance, ça marche encore". Le geste rentre mieux qu'une page de théorie.

Erreur classique

Mélanger `os.path.join` partout et `Path` au milieu, sans cohérence. Choisis `pathlib` pour les nouveaux scripts. Autre classique : oublier que le dossier courant n'est pas toujours le dossier du script. Utilise `Path(__file__).parent` quand le fichier doit être "à côté" du code. Autre piège : oublier `encoding="utf-8"` et voir des caractères bizarres dans un CSV français.

⚠ ATTENTION

Un chemin relatif "qui marche chez toi" peut planter demain si tu lances depuis un autre dossier. Ancre, ou passe le chemin en argument.

En vrai

Crée un dossier `data` à côté d'un script. Écris `bonjour.txt` avec `write_text`. Relis-le. Affiche le chemin absolu avec `resolve()`. Change de répertoire dans le terminal et relance : si tu as bien ancré le chemin sur `__file__`, ça continue de marcher.

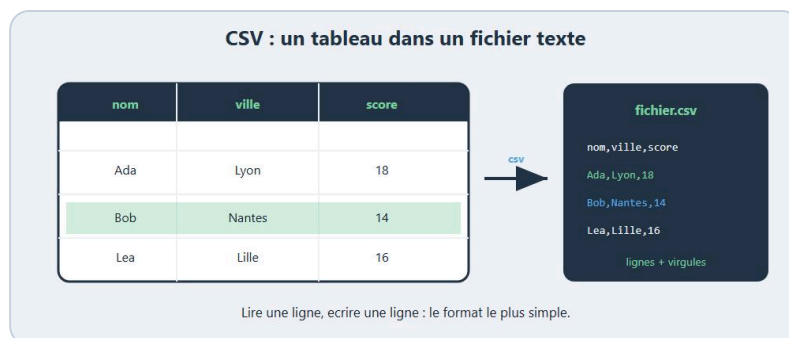
A toi

Écris un petit script qui crée `data/notes.csv` vide (ou avec une ligne d'en-tête `eleve,matiere,note`), en utilisant uniquement `Path`. Pas de `os.path`. Le geste compte : on branchera le vrai CSV au chapitre suivant. Bonus : affiche `resolve()` du fichier créé pour vérifier où il vit vraiment sur le disque.

★ A RETENIR

Un chemin, c'est un objet `Path`. Assemble avec `/`, ancre avec `__file__` quand le fichier vit à côté du script.

Chapitre 3 - CSV : lire et ecrire un tableau



CSV : un tableau dans un fichier texte.

Un fichier **CSV**, c'est un tableau en texte. Chaque ligne est une ligne du tableau. Les colonnes sont separees par une virgule (parfois un point-virgule en France). Excel et les tableurs adorent ce format. Les scripts aussi. C'est le pont le plus simple entre Python et le monde "metier" : notes d'eleves, listes clients, exports comptables. Tu n'as pas besoin d'un tableur ouvert a minuit pour calculer une moyenne.

Tu pourrais decouper a la main avec `.split(",")`. Ca marche jusqu'au jour ou une cellule contient une virgule, ou des guillemets. Le module `csv` de la bibliotheque standard gere ca proprement. Un CSV, c'est un tableur aplati. La premiere ligne dit "voici les noms de colonnes". Chaque ligne suivante est une ligne de donnees. Python lit ligne par ligne et te donne des dictionnaires avec des cles nommees. Tu penses en colonnes (`eleve`, `matiere`, `note`), pas en indices `0`, `1`, `2`. C'est plus lisible, et plus solide quand l'ordre des colonnes change.

Chez DanielCraft, on dit : pour un tableau, utilise l'outil tableau. Sam lit les notes de ses eleves avec **DictReader**. Lea exporte des moyennes vers un second CSV pour ses clients. Max recoit des factures fournisseurs en CSV et les resume sans ouvrir Excel. Sam a six lignes : Alice et Bob, maths et francais. DictReader lui donne `ligne["eleve"]`, `ligne["note"]`. Il calcule, il exporte. Zero copier-coller.

Un exemple concret : notes

Imagine `data/notes.csv` :

```
eleve,matiere,note
Alice,maths,14
Bob,maths,11
Alice,francais,16
Bob,francais,13
```

Premiere ligne : les noms de colonnes (en-tete). Ensuite les donnees. Simple, lisible, partageable.

Lire avec csv.DictReader

```
import csv
from pathlib import Path

chemin = Path("data/notes.csv")

with chemin.open(encoding="utf-8", newline="") as f:
    lecteur = csv.DictReader(f)
    for ligne in lecteur:
        print(ligne["eleve"], ligne["matiere"], ligne["note"])
```

Chaque `ligne` est un dictionnaire. Les clés viennent de l'en-tête. Le `newline=""` est une recommandation officielle sous Windows pour le module `csv`. Garde-le. L'`encoding="utf-8"` protège les accents.

Attention : les valeurs restent des **chaînes**. `"14"` n'est pas encore le nombre `14`. Convertis quand tu calcules :

```
note = float(ligne["note"])
```

♦ ASTUCE

Des le premier calcul, force le `float`. Tu évites le classique `"14" + "16"` qui donne `"1416"` au lieu de `30`.

Ecrire avec csv.DictWriter

```
import csv
from pathlib import Path

chemin = Path("data/notes.csv")
lignes = [
    {"eleve": "Alice", "matiere": "maths", "note": "14"},
    {"eleve": "Bob", "matiere": "maths", "note": "11"},
]

with chemin.open("w", encoding="utf-8", newline="") as f:
    champs = ["eleve", "matiere", "note"]
    ecriture = csv.DictWriter(f, fieldnames=champs)
    ecriture.writeheader()
    ecriture.writerows(lignes)
```

`writeheader()` écrit la première ligne. `writerows` écrit le reste. Tu peux aussi faire `writerow` une ligne à la fois dans une boucle.

Séparateur point-virgule

En France, Excel exporte parfois avec `;`. Dis-le au lecteur :

```
lecteur = csv.DictReader(f, delimiter=";")
```

Et à l'écriture :

```
ecriture = csv.DictWriter(f, fieldnames=champs, delimiter=";")
```

Si tu ouvres un CSV et que "tout est dans une seule colonne", regarde le separateur.

⚠ ATTENTION

Tout est string jusqu'a conversion. `"14" + "16"` donne `"1416"`, pas `30`. Passe par `float` avant de calculer.

Calculer une moyenne

Tu lis, tu accumules, tu divises. Rien de magique :

```
total = 0.0
compte = 0
with chemin.open(encoding="utf-8", newline="") as f:
    for ligne in csv.DictReader(f):
        if ligne["eleve"] == "Alice":
            total += float(ligne["note"])
            compte += 1
moyenne = total / compte if compte else 0
print("Moyenne Alice :", moyenne)
```

C'est le coeur de beaucoup de petits scripts "metier" : lire un tableau, filtrer, resumer.



Exemple : lignes brutes, puis un resume clair.

Petite histoire

Sam exportait les notes depuis son logiciel scolaire en CSV. Il ouvrait Excel, faisait des moyennes, imprimait. Un soir, une formule s'était cassée et personne ne l'avait vu. Avec un script Python de trente lignes, il relance la même commande chaque trimestre. Les moyennes sont identiques à la main, mais vérifiables. Lea fait pareil pour ses rapports clients : un CSV entrée, un CSV sortie, zero copier-coller.

Max a adapté le même pattern à ses factures fournisseurs. Colonnes différentes, même geste. C'est ça la force du CSV : un format bête, un pont solide.

Erreur classique

Ouvrir en mode texte sans `newline=""`. Ou oublier que tout est string et calculer `"14" + "16"` au lieu de `14 + 16`. Ou écrire sans `writeheader` puis s'étonner que `DictReader` ne trouve pas les clés. Autre piège : confondre le nom du fichier et son contenu - un mauvais chemin donne un fichier vide ou une erreur, pas un "CSV cassé".

En vrai

Crée `notes.csv` à la main (bloc-notes) avec 4-5 lignes. Lis-le avec `DictReader`. Affiche la moyenne d'un élève. Puis réécris un nouveau fichier `moyennes.csv` avec deux colonnes `eleve, moyenne`.

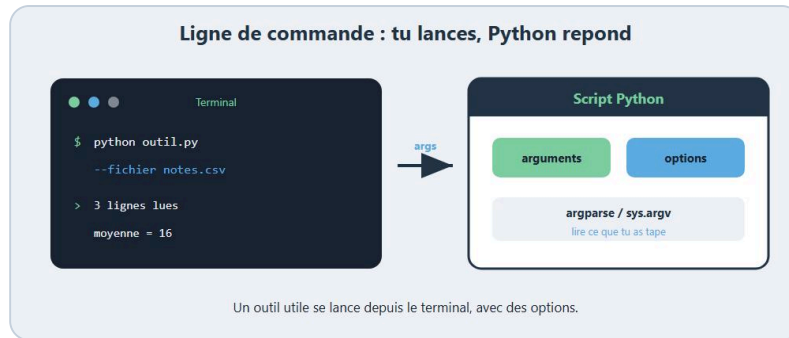
A toi

Écris une fonction `moyenne_eleve(chemin, nom)` qui retourne la moyenne, ou `None` si l'élève est absent. Teste-la avec ton fichier. On réutilisera ce genre de fonction dans le mini-projet et les ateliers. Bonus : affiche aussi le nombre de notes prises en compte. Sam vérifie toujours à la main sur un élève avant de faire confiance au script.

★ A RETENIR

CSV = tableau texte. `DictReader` / `DictWriter`, UTF-8, `newline=""`, et float avant de calculer.

Chapitre 4 - argparse : un script avec des arguments



Un script que tu lances avec des options.

Tu lances Python avec `python mon_script.py`. C'est bien. Encore mieux : `python mon_script.py --fichier data/notes.csv --eleve Alice`. Le script lit des options. Tu n'as plus à modifier le code pour changer le fichier ou le nom. C'est la différence entre un brouillon et un **outil**.

`argparse` fait partie de la bibliothèque standard. Il parse la ligne de commande, affiche une aide (`-h`), et refuse les options invalides. Ton programme déclare : "j'accepte un fichier, et optionnellement un eleve". L'utilisateur passe ces infos en tapant dans le terminal. Toi, tu reçois un objet propre avec des attributs. Plus besoin de fouiller `sys.argv` à la main (sauf cas très simples). Le **CLI** n'est pas le cœur métier. C'est la poignée de la porte. La logique (moyenne, filtre) vit dans des fonctions. `argparse`, lui, ouvre la porte.

Chez DanielCraft, on aime les scripts qui s'expliquent tout seuls. Lea lance `moyenne.py --eleve Alice` sans ouvrir l'éditeur. Max passe `--fichier factures.csv` depuis n'importe quel dossier. Sam montre `-h` à un collègue et celui-ci comprend sans lire le source.

♦ ASTUCE

Ecris l'aide `-h` comme si tu parlais à ton futur toi fatigué. Si l'aide est claire, le script survit.

Premier exemple

```
import argparse

parser = argparse.ArgumentParser(description="Resume des notes CSV")
parser.add_argument("--fichier", required=True, help="Chemin vers le CSV")
parser.add_argument("--eleve", default=None, help="Filtrer sur un eleve")
args = parser.parse_args()

print("Fichier :", args.fichier)
print("Eleve :", args.eleve)
```

Lance :

```
python resume.py --fichier data/notes.csv --eleve Alice
```

Ou demande l'aide :

```
python resume.py -h
```

Tu verras la description et les options. C'est déjà "pro" pour un petit outil.

Types et drapeaux

Tu peux demander un entier, un flottant, un booléen style drapeau :

```
parser.add_argument("--seuil", type=float, default=10.0)
parser.add_argument("--verbose", action="store_true", help="Plus de details")
```

Avec `--verbose`, `args.verbose` vaut `True`. Sans, `False`. Avec `--seuil 12`, tu reçois `12.0`. Si quelqu'un met du texte à la place d'un nombre, `argparse` proteste clairement.

Relier au CSV

```
import argparse
import csv
from pathlib import Path

parser = argparse.ArgumentParser(description="Moyenne d'un eleve")
parser.add_argument("--fichier", required=True)
parser.add_argument("--eleve", required=True)
args = parser.parse_args()

chemin = Path(args.fichier)
total = 0.0
compte = 0
with chemin.open(encoding="utf-8", newline="") as f:
    for ligne in csv.DictReader(f):
        if ligne["eleve"] == args.eleve:
            total += float(ligne["note"])
            compte += 1

if compte == 0:
    print("Aucune note pour", args.eleve)
else:
    print(f"Moyenne de {args.eleve} : {total / compte:.1f}")
```

Tu sens le pattern : parser les arguments, puis faire le travail.

Arguments positionnels

Parfois tu veux `python outil.py notes.csv` sans `--fichier` :

```
parser.add_argument("fichier", help="Chemin CSV")
```

Pas de tirets. L'ordre compte. Mélange positionnels et options selon ce qui est le plus naturel à taper. Lea préfère souvent des options nommées (`--fichier`) pour les scripts partagés : on lit l'appel sans se souvenir de l'ordre. Max aime un positionnel quand il tape vite le matin. Les deux sont valides. Choisis selon l'usage, pas selon la mode.

Ce que ce n'est pas

argparse, ce n'est pas un framework web. Ce n'est pas non plus une excuse pour tout mettre dans `main`. Ce n'est pas obligatoire d'avoir vingt options dès le premier script. Deux options claires battent dix options floues. Et ce n'est surtout pas "parser `sys.argv` à la main pour faire plus pro" : argparse fait déjà l'aide et la validation.



Exemple : lancer le script avec `--fichier`.

Petite histoire

Max avait trois copies de son script : un pour Alice, un pour Bob, un pour le fichier de l'année dernière. Chaque changement, il dupliquait. Lea lui a montré argparse en dix minutes. Un seul fichier, trois lancements différents. Max a supprimé les doublons le jour même. C'est ça la valeur d'un CLI propre : moins de fichiers morts, moins de confusion.

Sam, en salle des profs, a montré `-h` à un collègue. Le collègue a lancé le script sans ouvrir le code. Mission réussie. Chez DanielCraft, c'est le test : si quelqu'un comprend avec `-h` seul, tu as un outil.

Erreur classique

Tout mettre en dur dans le code (`fichier = "notes.csv"`) et dupliquer le script pour chaque cas. Ou parser `sys.argv[1]` sans aide ni vérification. Autre classique : oublier `required=True` puis planter plus loin avec un `None` mystérieux. Mieux vaut une erreur argparse dès le début. Autre piège : écrire une aide vague ("fichier : un fichier") au lieu de préciser le format attendu.

⚠ ATTENTION

Si une option est obligatoire, mets `required=True`. Une erreur argparse claire bat un `None` mystérieux au milieu du calcul.

En vrai

Reprends ton lecteur de notes. Ajoute argparse. Lance-le trois fois avec des élèves différents, sans toucher au code. Affiche `-h` et lis ce que ça dit. Si l'aide est claire, ton futur toi (ou un collègue) te remerciera.

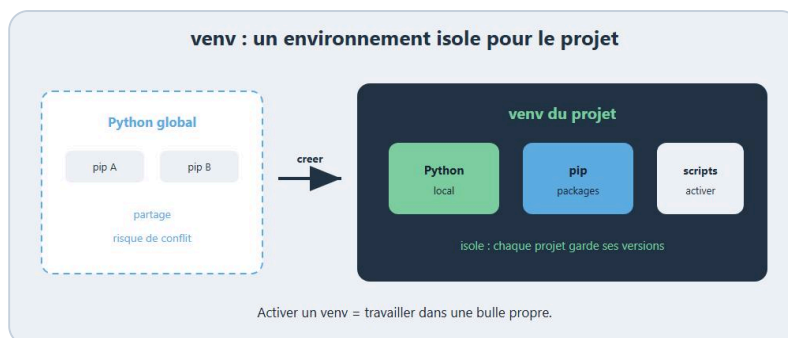
A toi

Ajoute une option `--seuil` : n'affiche que les notes au-dessus du seuil, ou un message "aucune note au-dessus". Petit, utile, concret. C'est exactement le genre d'outil qu'on garde sous le coude chez DanielCraft.

★ A RETENIR

argparse transforme un brouillon en outil : options, aide `-h`, refus clair des mauvaises saisies.

Chapitre 5 - venv et pip : isoler les paquets



venv : un coin isole pour tes paquets.

Python de base sait beaucoup de choses. Mais pour parler HTTP facilement, on utilisera `requests` (chapitre suivant). `requests` n'est pas dans la bibliotheque standard : tu l'installes avec `pip`.

Si tu installes tout "dans le Python global" de ta machine, les projets se marchent dessus. Une version de bibliotheque pour le projet A casse le projet B. La solution : un environnement virtuel (**venv**) par projet. Un venv, c'est une boite a outils privée pour un projet. Dedans : un Python isole et un pip isole. Quand tu actives la boite, tes commandes `python` et `pip` pointent vers elle. Quand tu desactives, tu reviens au Python systeme. Rien ne se melange. Tu changes de dossier, tu changes de boite.

Chez DanielCraft, regle simple : un projet, un venv. Lea a deux clients avec deux versions de `requests` differentes. Max a un script meteo isole de son outil CSV. Sam recree son venv proprement chaque annee scolaire.

★ A RETENIR

Un projet, un venv. On partage `requirements.txt` , pas le dossier `.venv` .

Creer un venv

Dans le dossier de ton projet :

```
python -m venv .venv
```

Ca cree un dossier `.venv` (le nom est une convention courante). Dedans : un Python isole et un `pip` isole.

L'activer

Sous Windows (PowerShell) :

```
.\.venv\Scripts\Activate.ps1
```

Sous Windows (cmd) :

```
.\.venv\Scripts\activate.bat
```

Sous macOS / Linux :

```
source .venv/bin/activate
```

Quand c'est actif, ton invite de commande montre souvent `(.venv)`. Les commandes `python` et `pip` pointent alors vers le venv. Pour desactiver : `deactivate`.

Si PowerShell refuse l'execution de scripts, c'est une politique Windows. Tu peux chercher la doc officielle sur l'execution de scripts, ou utiliser `cmd` pour activer. L'idée reste la meme.

Installer un paquet

Avec le venv actif :

```
pip install requests
```

Tu peux figer les versions dans un fichier `requirements.txt` :

```
requests==2.32.3
```

Puis installer d'un coup :

```
pip install -r requirements.txt
```

On approfondira l'organisation du projet plus loin. Pour l'instant, retiens : `pip install` dans le venv, pas "au feeling" dans le Python systeme.

Verifier

```
python -c "import requests; print(requests.__version__)"
```

Si ca affiche une version, `requests` est bien la. Si tu desactives le venv et que tu relances sans le paquet global, ca peut echouer - et c'est normal.

.gitignore

Le dossier `.venv` ne se partage pas sur Git. Il est lourd et propre a chaque machine. On partage `requirements.txt`, et chacun recree son venv. Si tu utilises Git, ajoute `.venv/` au `.gitignore`.

Mettre a jour pip (souvent utile)

Dans le venv fraichement cree, pip peut etre vieux. Un classique :

```
python -m pip install --upgrade pip
```

Puis installe tes paquets. Moins de messages bizarres, moins de surprises.

Plusieurs projets, plusieurs venv

Projet notes d'un cote, projet meteo de l'autre : deux dossiers, deux `.venv`. Tu actives celui du projet sur lequel tu travailles. Si tu changes de dossier dans le terminal, pense a reactiver (ou a ouvrir un terminal "dans" le bon projet via ton editeur).

♦ ASTUCE

Toujours `python -m pip install ...` avec le meme interpreteur. Comme ca, pip et python sont maries. Fini le "j'ai installe mais ModuleNotFoundError".

Petite histoire

Lea avait installe `requests` "quelque part" sur sa machine. Son script marchait chez elle, pas chez le client. Le diagnostic : elle avait un `pip` et un `python` qui ne parlaient pas au meme endroit. La regle `python -m pip install requests` a tout regle en une commande. Depuis, chaque projet a son venv et son `requirements.txt`. Zero surprise entre machines.

Max a perdu une soiree a chercher pourquoi "ca marchait hier". Sam lui a demande : "venv active ?" Non. Relance, ca marche. Le rituel compte autant que la commande.

Erreur classique

Installer avec un `pip` et lancer avec un autre `python`. Symptome : "j'ai installe requests mais Python dit ModuleNotFoundError". Contre-mesure : toujours `python -m pip install ...` avec le meme interpreteur.

```
python -m pip install requests
```

Comme ca, pip et python sont maries. Autre classique : oublier d'activer le venv et croire que "pip est casse". Ou supprimer le dossier `.venv` a la main puis s'etonner que plus rien ne s'importe : il suffit de recreer le venv et de reinstaller depuis `requirements.txt`.

⚠ ATTENTION

Ne committe jamais `.venv`. Il est lourd, propre a chaque machine, et inutile sur Git. Committe `requirements.txt`.

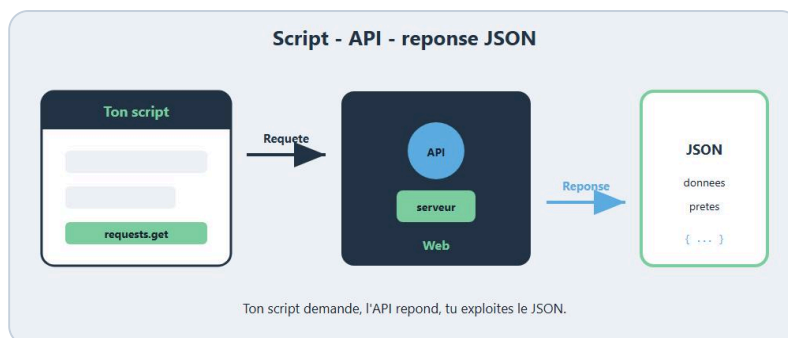
En vrai

Cree un dossier `demo-venv`. Fais `python -m venv .venv`. Active. Installe `requests`. Verifie l'import. Desactive. C'est un rituel a connaitre par coeur.

A toi

Ecris un `requirements.txt` minimal avec `requests`. Documente en trois lignes (dans un `README` perso ou un commentaire) : creer le venv, activer, installer. Ton futur toi aime les rituels ecrits. Bonus : recree le venv depuis zero sur une machine (ou un dossier) propre en suivant uniquement ton README. Si tu bloques, le rituel est trop flou : corrige-le avant d'ajouter un paquet.

Chapitre 6 - requests GET : appeler une API JSON



Ton script demande des données à une API.

Jusqu'ici, tes données vivaient dans des fichiers. Maintenant, ton script peut demander des infos ailleurs : une **API** météo, un service public, un endpoint de demo. Le réseau entre dans ton quotidien Python. C'est un cran de plus : plus seulement "lire un CSV", mais "demander à un serveur et utiliser la réponse".

requests est une bibliothèque très utilisée pour ça. En mode **GET**, tu dis : "donne-moi cette ressource". Une API, c'est un serveur qui répond à des questions via HTTP. Tu envoies une URL (avec parfois des paramètres). Tu reçois un code status (200 = OK en général) et un corps (souvent du JSON). Ton script extrait ce dont il a besoin et affiche un résumé. Pas besoin d'interface graphique pour être utile. Une phrase dans le terminal, le matin, ça suffit.

Chez DanielCraft, on résume : demander, lire, utiliser. Max check la météo avant un chantier extérieur. Lea récupère des données publiques pour un dashboard client. Sam montre à ses élèves comment une URL renvoie du **JSON** lisible par Python.

★ A RETENIR

Garde le pattern en tête : GET -> status -> `.json()` -> extraire deux-trois champs -> afficher clair.

Installer (rappel)

Dans ton venv :

```
python -m pip install requests
```

Premier appel

```
import requests

url = "https://httpbin.org/json"
reponse = requests.get(url, timeout=10)
print(reponse.status_code)
data = reponse.json()
print(data)
```

Que se passe-t-il ? `requests.get` part chercher l'URL. Tu reçois un objet `reponse`. `status_code` est le code HTTP (200 = OK en général). `.json()` parse le corps si c'est du JSON, et te donne un dict ou une liste Python.

Le `timeout=10` evite d'attendre indefiniment si le reseau rame. On en reparlera au chapitre erreurs. Prends l'habitude de le mettre des maintenant.

Meteo : exemple concret

Beaucoup d'API meteo renvoient un JSON du genre :

```
{ "ville": "Paris", "temp": 18.2, "ciel": "nuageux" }
```

Ton script fait un GET sur une URL avec la ville en parametre, lit le JSON, puis affiche une phrase. Memes etapes partout. Seules les cles changent.

Avec Open-Meteo (API publique, sans cle pour un usage simple), l'idee ressemble a :

```
import requests

url = "https://api.open-meteo.com/v1/forecast"
params = {
    "latitude": 48.85,
    "longitude": 2.35,
    "current_weather": "true",
}
reponse = requests.get(url, params=params, timeout=10)
data = reponse.json()
meteo = data["current_weather"]
print("Temperature :", meteo["temperature"])
```

`params` ajoute les parametres a l'URL proprement (encodage inclus). Tu n'as pas a coller `?latitude=...` a la main.

Afficher un resume

```
temp = meteo["temperature"]
vent = meteo["windspeed"]
print(f"Il fait {temp} C, vent {vent}")
```

Tu vois le pattern de beaucoup d'outils CLI : appeler, extraire deux-trois champs, afficher clairement.

Headers (rapide)

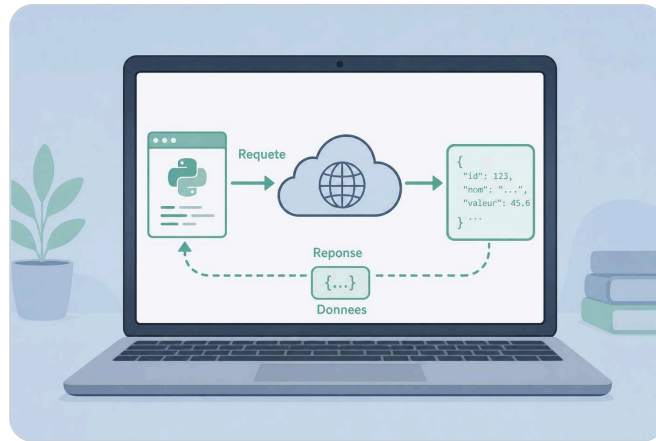
Parfois une API demande un en-tete (type de contenu, jeton...). Avec requests :

```
headers = {"Accept": "application/json"}
reponse = requests.get(url, headers=headers, timeout=10)
```

On gardera les secrets (jetons) pour le chapitre variables d'environnement. Ne colle jamais une cle API en dur dans un fichier que tu partages.

⚠ ATTENTION

Pas de `timeout` = script qui peut attendre indefiniment. Mets `timeout=` des le premier GET. Toujours.



Exemple : requete, reponse, donnees.

Petite histoire

Max ouvrait trois onglets meteo chaque matin avant de partir en toiture. Lea lui a montre un script de quinze lignes avec Open-Meteo. Un lancement, une phrase : "Il fait 12 C, vent 25". Max l'utilise encore. Ce n'est pas revolutionnaire. C'est juste un outil qui reste sous le coude parce qu'il sert vraiment.

Sam a fait la demo en cours : URL, JSON, temperature. Les eleves ont vu "le reseau" devenir quelque chose de concret, pas un nuage abstrait. Chez DanielCraft, c'est le genre de demo qui ancre.

Erreur classique

Oublier `.json()` et manipuler `reponse.text` en croyant que c'est deja un dict. Ou appeler sans timeout. Ou croire que "pas d'exception" signifie "tout va bien" : un 404 ne leve pas toujours une exception avec requests (selon comment tu codes). Le prochain chapitre traite ca.

En vrai

Dans ton venv, appelle une URL JSON publique (httpbin, open-meteo, ou autre dispo chez toi). Affiche le `status_code` et deux cles du JSON. Le but : voir de vraies donnees arriver dans ton terminal. Si ca rame, baisse le timeout et observe. Si ca marche, souris : ton script parle au monde.

A toi

Ecris une fonction `temperature_paris()` qui retourne la temperature actuelle (float), ou leve une erreur claire si le JSON n'a pas la forme attendue. On solidifiera les erreurs juste apres. Bonus : affiche aussi le vent dans une phrase complete, style Max le matin. Garde le timeout des maintenant, meme si le chapitre erreurs vient juste apres : le reflexe se construit tot.

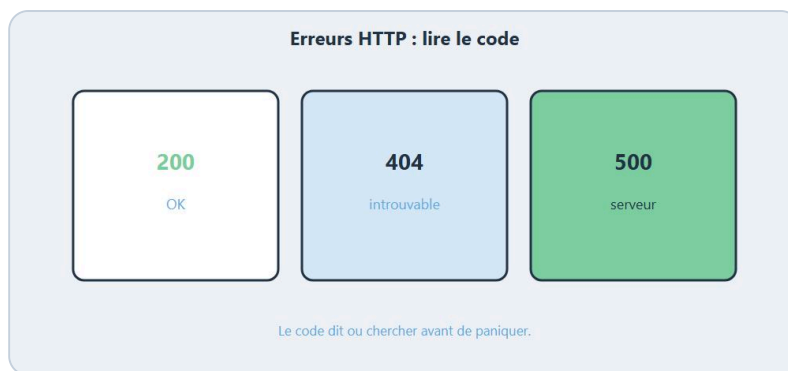
Zoom : API = contrat

Une API, c'est un contrat. Tu demandes une forme. Tu recois une forme. Si la forme change, ton script casse. Lea lit toujours un exemple de JSON avant de coder. Sam projette le JSON au tableau. Max regarde deux cles seulement et ignore le reste. Chez DanielCraft, on extrait peu, clairement, plutot que de tout imprimer "pour voir" en production.

♦ **ASTUCE**

GET + timeout + `.json()` + extraire clairement. Le reseau devient un outil, pas un mystere.

Chapitre 7 - Erreurs HTTP : status, timeout, try/except



200 OK, 404 introuvable, 500 serveur : lire le code.

Appeler le reseau, c'est accepter que ca casse. Pas de wifi. Serveur en panne. URL fausse. Reponse trop lente. JSON inattendu. Un script "pratique" gere ca sans afficher une page de traceback a l'utilisateur. C'est la difference entre un demo et un **outil**.

Chaque appel reseau a trois sorties possibles : succes (200 + JSON ok), echec **HTTP** (404, 500...), echec reseau (**timeout**, pas de connexion). Ton script doit traiter les trois sans paniquer. L'utilisateur voit une phrase humaine. Toi, tu as le detail pour deboguer. Deux canaux, une meme honnetete.

Chez DanielCraft, regle d'or : echec possible = message clair + details techniques a part (log ou console). Lea affiche "Impossible de charger les donnees" a son client et garde le detail HTTP dans un log. Max voit "Serveur trop lent" au lieu d'un mur d'erreur. Sam montre a ses eleves que le reseau, ca casse, et que Python sait le gerer proprement.

⚠ ATTENTION

N'avale jamais toutes les exceptions avec `except: pass`. Tu caches le probleme. Message clair + detail a part, pas le silence.

status_code et raise_for_status

```
import requests

reponse = requests.get(url, timeout=10)
print(reponse.status_code)

if reponse.status_code != 200:
    print("Echec HTTP :", reponse.status_code)
else:
    data = reponse.json()
```

Plus court, style requests :

```
reponse = requests.get(url, timeout=10)
reponse.raise_for_status() # leve une erreur si 4xx ou 5xx
data = reponse.json()
```

`raise_for_status()` transforme un 404 ou 500 en exception. Tu peux la capturer dans un `try/except`.

Timeout

```
try:
    reponse = requests.get(url, timeout=5)
    reponse.raise_for_status()
    data = reponse.json()
except requests.Timeout:
    print("Le serveur met trop de temps a repondre.")
except requests.HTTPError as e:
    print("Erreur HTTP :", e.response.status_code)
except requests.RequestException:
    print("Impossible de contacter le serveur.")
```

`RequestException` est la famille large (reseau, HTTP, etc.). Commencer large est OK pour un petit script. Affiner ensuite.

Tu peux aussi ecrire `timeout=(3, 10)` : 3 secondes pour connecter, 10 pour lire. Pour debuter, un seul nombre suffit.

JSON pourri

Parfois le status est 200, mais le corps n'est pas du JSON, ou il manque une cle.

```
try:
    data = reponse.json()
    temp = data["current_weather"]["temperature"]
except ValueError:
    print("La reponse n'est pas du JSON valide.")
except KeyError:
    print("JSON inattendu : cle manquante.")
```

Ne suppose pas que l'API est parfaite. Verifie ce dont tu as besoin.

Pattern solide pour un mini outil

```
def fetch_json(url, params=None):
    try:
        r = requests.get(url, params=params, timeout=10)
        r.raise_for_status()
        return r.json()
    except requests.Timeout:
        raise RuntimeError("Delai depasse") from None
    except requests.RequestException as e:
        raise RuntimeError(f"Echec reseau : {e}") from None
```

Tu centralises. Le reste du programme appelle `fetch_json` et gere un message utilisateur. Moins de copier-coller.

♦ ASTUCE

Centralise le fetch dans une fonction. Le CLI n'a qu'a afficher "Impossible de..." et logger le detail. Moins de doublons, plus de clarte.

Petite histoire

Lea a deploye un script meteo chez un client sans gestion d'erreur. Un jour, l'API a renvoye un 503 pendant dix minutes. Le client a reçu une page de traceback Python par email. Lea a ajoute timeout, `raise_for_status`, et des messages clairs le soir meme. Depuis, les echecs reseau sont visibles mais propres. C'est ca la maturite d'un script pratique.

Max a teste avec un timeout de 0.001 seconde juste pour voir. Message propre. Sam le fait faire en classe : "casse, puis reparer". Le reflexe rentre.

Erreur classique

Avaler toutes les exceptions avec `except:` vide (ou `except Exception: pass`). Tu caches le probleme au lieu de le comprendre. Autre classique : afficher le traceback brut a un utilisateur non technicien. Garde le detail pour toi (`print` debug ou logging au chapitre 9). Autre piege : oublier le timeout et bloquer un script automatise pendant des heures.

En vrai

Appelle une URL volontairement fausse. Puis une URL lente avec un timeout tres court. Observe les exceptions. Branche des messages humains. Casse, puis reparer : c'est comme ca qu'on apprend le reseau. Note sur papier les trois messages que tu affiches. S'ils sont clairs pour Max non technicien, tu as gagne.

A toi

Reprends `temperature_paris()` (ou equivalent). Ajoute timeout, `raise_for_status`, et des messages clairs pour timeout / HTTP / JSON. Si les trois chemins d'echec sont propres, ce chapitre est reussi. Bonus : centralise dans `fetch_json` et reutilise. Max teste souvent avec `timeout=0.001` juste pour voir le message. Toi aussi : casse volontairement avant de dire "c'est solide".

Zoom : deux canaux

L'utilisateur lit une phrase. Toi, tu lis un log. Si tu melanges les deux, tu noies le client ou tu te prives de details. Lea affiche "Impossible de charger les donnees" et loggue le status. Sam oblige les eleves a ecrire les deux phrases avant de coder. Chez DanielCraft, c'est la signature d'un outil adulte.

★ A RETENIR

Succes, echec HTTP, echec reseau : trois sorties, messages humains, details a part.

Chapitre 8 - Variables d'environnement



.env local, os.environ au runtime. Pas de secrets commits.

Une cle API, un mot de passe, un jeton : ca ne doit pas vivre en clair dans ton code source. Surtout si tu partages le projet, ou si tu le pousse sur Git. La bonne habitude : lire les **secrets** depuis l'environnement.

`os.environ` donne acces aux variables d'environnement du systeme (ou de ton terminal). Le code dit : "j'ai besoin d'une variable nommee `API_CLE`". La valeur vient de l'exterieur : le terminal, un fichier `.env`, ou le systeme. Si la variable manque, le script s'arrete avec un message clair. Personne ne voit le secret dans le source partage. Tu peux montrer ton code. Tu ne montres pas ta cle.

Chez DanielCraft, on dit : le code connait le nom de la variable, pas la valeur secrete. Lea stocke sa cle API meteo dans `API_CLE`, jamais dans le fichier. Max a un fichier `.env` local ignore par Git. Sam montre a ses eleves la difference entre config publique et secret.

⚠ ATTENTION

Une cle poussee sur GitHub est brulee. Revoque-la. Puis `.env` + `.gitignore` + `.env.example`. Pas de "juste cette fois".

Lire une variable

```
import os

cle = os.environ.get("API_CLE")
if not cle:
    raise RuntimeError("Variable API_CLE manquante")
```

`.get` retourne `None` si absente. Tu peux aussi ecrire `os.environ["API_CLE"]`, qui leve une `KeyError` si elle n'y est pas. Pour un script clair, verifier et afficher un message explicite est souvent mieux.

Definir la variable avant de lancer

Sous Windows PowerShell :

```
$env:API_CLE = "valeur-secrete"
python mon_script.py
```

Sous cmd :


```
set API_CLE=valeur-secrete
python mon_script.py
```

Sous macOS / Linux :

```
export API_CLE=valeur-secrete
python mon_script.py
```

La variable existe pour cette session de terminal. Ce n'est pas magique dans le fichier `.py`.

Le fichier `.env`

Beaucoup de projets utilisent un fichier `.env` à la racine :

```
API_CLE=valeur-secrete
VILLE_DEFAULT=Paris
```

Ce fichier n'est pas du Python. Des outils (comme la bibliothèque `python-dotenv`) peuvent le charger dans `os.environ` au démarrage. L'idée importante, même sans installer quoi que ce soit pour l'instant :

Tu ne commits pas `.env` (secrets). Tu commits un `.env.example` avec les noms de variables et des fausses valeurs.

```
API_CLE=remplace-moi
VILLE_DEFAULT=Paris
```

Comme ça, un collègue sait quoi configurer sans voir tes vrais secrets.

Charger `.env` à la main (mini version)

Pour comprendre le principe, sans dépendance :

```
from pathlib import Path
import os

def charger_env(chemin=".env"):
    p = Path(chemin)
    if not p.exists():
        return
    for ligne in p.read_text(encoding="utf-8").splitlines():
        ligne = ligne.strip()
        if not ligne or ligne.startswith("#") or "=" not in ligne:
            continue
        nom, valeur = ligne.split("=", 1)
        os.environ.setdefault(nom.strip(), valeur.strip())
```

`setdefault` ne recouvre pas une variable déjà définie dans le système. Utile.

En pratique, pour un vrai projet, `python-dotenv` fait ça plus complètement. L'essentiel est le réflexe : secrets dehors du code.

Config non secrete

Tout n'est pas secret. Une ville par default, un seuil, un mode debug : variables d'environnement ou arguments CLI, selon ce qui est le plus pratique. Les secrets, eux, passent presque toujours par l'environnement (ou un gestionnaire de secrets plus avance, hors sujet ici).

Ce que ce n'est pas

Les variables d'environnement, ce n'est pas "cacher le code". Ce n'est pas non plus une raison de mettre toute la config dans `.env` sans reflechir. Ce n'est pas magique : si la variable n'est pas definie dans la session qui lance le script, Python ne la invente pas. Et ce n'est surtout pas une excuse pour logger la valeur "juste pour verifier".

♦ ASTUCE

Meme si ton API demo n'a pas de cle, prends le reflexe `.env.example` maintenant. Le muscle se construit avant l'urgence.

Petite histoire

Lea a pousse un script sur GitHub avec `API_CLE = "sk-xxxxx"` en haut du fichier. Quelqu'un a scanne le depot en dix minutes. La cle etait brulee. Revocation, nouvelle cle, `.env.example`, `.gitignore` mis a jour. Depuis, c'est un reflexe chez DanielCraft : jamais de secret dans le code, jamais dans un screenshot, jamais dans un log.

Max a failli logger sa cle "pour debug". Sam l'a arrete. Une ligne de log, une cle exposee. Pas ce soir.

Erreur classique

Ecrire `API_CLE = "sk-xxxxx"` en haut du fichier et pousser sur GitHub. Une fois public, considere la cle brulee : revoque-la. Autre classique : logger la cle "pour debug". Ne jamais imprimer un secret. Autre piege : committer le fichier `.env` par erreur parce qu'il n'est pas dans `.gitignore`.

En vrai

Cree une variable `MON_NOM` dans ton terminal. Lis-la dans un script et affiche "Bonjour, ...". Puis simule l'absence de variable et affiche une erreur claire.

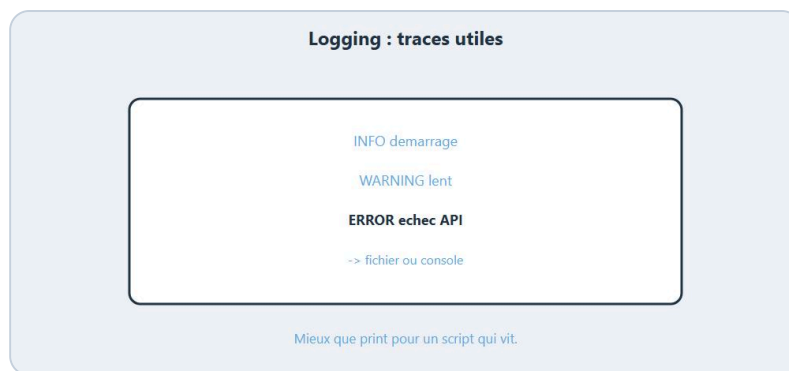
A toi

Prepares un `.env.example` pour un futur script meteo ou API (noms de variables seulement). Ajoute `.env` a ton `.gitignore` mental (et reel si tu utilises Git). Le muscle "pas de secret dans le code" se construit maintenant.

★ A RETENIR

Le code connait le nom de la variable. La valeur secrete reste dehors : environnement, `.env` ignore, `.env.example` partageable.

Chapitre 9 - logging basique



INFO, WARNING, ERROR : des traces utiles.

`print` est parfait pour apprendre. Pour un script qui vit un peu, `logging` est plus souple : niveaux (info, warning, error), format, et possibilité d'écrire dans un fichier sans changer toute la logique. Tu gardes le résultat clair à l'écran, et le détail technique à part.

Deux canaux. Le canal utilisateur (`print`) : ce que l'humain doit lire comme résultat. Le canal technique (`logging`) : ce qui explique le déroulement, les warnings, les échecs détaillés. Tu peux couper le bruit technique en changeant un `niveau`, sans toucher au résultat affiché. Même script, deux modes : quotidien et chasse au bug.

Chez DanielCraft, on garde `print` pour l'affichage utilisateur du CLI, et `logging` pour ce qui se passe "sous le capot". Lea affiche "Moyenne : 14.2" à l'écran et logue "Lecture de data/notes.csv" en coulisse. Max voit "Impossible de charger la météo" mais retrouve le status HTTP dans `app.log`. Sam sépare résultat élève et trace technique pour déboguer sans noyer la classe.

★ A RETENIR

Règle simple : résultat humain -> `print`. Déroulement / échec technique -> `logging`. Si tu mélanges, tu noies le vrai message.

Premier logger

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s - %(message)s",
)

logging.info("Démarrage du script")
logging.warning("Fichier presque vide")
logging.error("Échec de l'appel API")
```

Tu verras des lignes avec le niveau. Si tu mets `level=logging.WARNING`, les `info` disparaît. Pratique : en prod tu baisses le bruit, en debug tu montes le détail.

Niveaux utiles

DEBUG : detail pour developper. **INFO** : deroulement normal. **WARNING** : bizarre mais on continue. **ERROR** : ca a rate. **CRITICAL** : tres grave.

Pour debuter, **INFO** + **ERROR** suffisent souvent. Ajoute **DEBUG** quand tu chasses un bug.

Logger nomme

Dans un projet a plusieurs fichiers, on evite le logger racine partout. Pattern courant :

```
import logging

logger = logging.getLogger(__name__)

def moyenne(valeurs):
    logger.debug("Nb valeurs : %s", len(valeurs))
    if not valeurs:
        logger.error("Liste vide")
        return None
    return sum(valeurs) / len(valeurs)
```

%s et les arguments separes : le message n'est formate que si le niveau est actif. Habitude propre.

Configure **basicConfig** une seule fois, cote point d'entree (**main**), pas dans chaque module.

Ecrire dans un fichier

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s",
    filename="app.log",
    encoding="utf-8",
)
```

Utile pour un script lance la nuit, ou pour relire apres coup. Tu peux aussi cumuler console + fichier avec des handlers (un cran plus avance : regarde la doc quand tu en auras besoin).

Avec requests / CSV

```
logger.info("Lecture de %s", chemin)
logger.info("Appel API %s", url)
logger.error("HTTP %s sur %s", status, url)
```

Tu retraces l'histoire sans noyer l'utilisateur. Sur le CLI, toi tu affiches : "Impossible de charger la meteo."

Dans le log : le status, l'URL, peut-etre l'exception.

print ou logging ?

Regle simple pour un CLI : ce que l'humain doit lire comme resultat va dans **print** (la moyenne, la temperature). Ce qui explique le deroulement ou l'echec technique va dans **logging**. Tu peux tout mettre en **print** sur un script de 20 lignes. Des que ca grandit, tu seras content d'avoir separe.

Exception + log

```
try:
    data = fetch_json(url)
except RuntimeError as e:
    logging.exception("Echec fetch")
    print("Impossible de recuperer les donnees.")
```

`logging.exception` (dans un `except`) ajoute la traceback au log. L'utilisateur voit une phrase calme. Toi, tu as le detail dans `app.log` ou dans la console de debug.

⚠ ATTENTION

Ne logue jamais un secret (jeton, mot de passe). Et n'appelle pas `basicConfig` dans trois modules differents : un seul point d'entree.

Petite histoire

Max avait mis des `print("DEBUG: ...")` partout dans son script meteo. Un matin, il ne voyait plus la temperature dans le flot de messages. Lea lui a montre la separation print / logging. Maintenant, Max lance avec `level=INFO` au quotidien et monte en `DEBUG` seulement quand ca casse. Meme script, deux modes, zero confusion.

Sam utilise le meme schema en classe : resultat au tableau (print), detail technique dans le log pour apres le cours. Les eleves comprennent la difference sans jargon.

Erreur classique

Logger des secrets (jetons, mots de passe). Ou laisser des centaines de `DEBUG` bruyants sans pouvoir les couper. Ou melanger `print` debug partout au point de ne plus voir le vrai resultat utilisateur. Ou appeler `basicConfig` dans trois modules differents et ne plus comprendre quel format gagne.

En vrai

Ajoute trois logs a ton script CSV ou meteo : demarrage, succes, echec. Change le niveau INFO vers WARNING et observe ce qui reste.

A toi

Separe clairement : un `print` pour le resume final, des `logging.info/error` pour le deroulement. Si tu peux couper le bruit en changeant un seul `level=`, c'est gagne. Bonus : ecris aussi dans un fichier `app.log` une seule fois, puis coupe le bruit console. Lea livre souvent comme ca : client calme, log complet pour elle.

♦ ASTUCE

Print pour l'humain. Logging pour le deroulement. Un niveau pour couper le bruit.

Chapitre 10 - datetime pratique



`now()` + format lisible.

Les dates sont partout : horodatage d'un log, "dernière mise à jour", filtrer des lignes après une certaine date, afficher un jour lisible. Le module `datetime` de la bibliothèque standard couvre l'essentiel. Tu n'as pas besoin d'une thèse sur les fuseaux pour être utile demain matin. Tu as besoin de trois gestes : maintenant, formater, parser.

Une date, c'est un objet Python que tu peux comparer, additionner (avec `timedelta`), formater en texte (`strftime`), ou parser depuis un texte (`strptime`). Ne compare jamais des dates en string : `"9/7/2026"` et `"10/7/2026"` mentent en ordre alphabétique. Parse d'abord, compare ensuite. Lea visualise un calendrier pose sur le bureau : les objets date, c'est ça. Les chaînes, c'est l'étiquette colle dessus pour l'humain.

On reste pratique. Pas de théorie profonde sur les fuseaux horaires (c'est un monde à part). Juste de quoi être à l'aise au quotidien. Sam filtre les événements du mois dans un CSV. Max horodate ses exports factures. Lea affiche "mis à jour le ..." dans ses rapports clients. Trois usages, une même boîte à outils. Chez DanielCraft, on insiste : ne compare jamais des dates en string. Cette règle seule évite des bugs absurdes qui font perdre une après-midi.

Ce que ce n'est pas

`datetime`, ce n'est pas "gestion complète du temps mondial". Ce n'est pas non plus une raison de paniquer devant `%Y-%m-%d`. Ce n'est pas obligatoire d'apprendre tous les motifs d'un coup. Et ce n'est surtout pas interchangeable avec un tri alphabétique de chaînes "qui a l'air de marcher". Ça ment souvent.

⚠ ATTENTION

Comparer des dates en chaînes, c'est un piège classique. `"9/7"` passe après `"10/7"` en alphabétique. Parse en `date`, puis compare.

Maintenant

```
from datetime import datetime, date, timedelta

maintenant = datetime.now()
aujourd_hui = date.today()
print(maintenant)
print(aujourd_hui)
```

`date` = jour sans heure. `datetime` = jour + heure. Souvent tu as besoin de l'un ou de l'autre, pas des deux en meme temps. Max utilise `date` pour ses factures. Lea utilise `datetime` pour ses logs. Sam montre les deux le meme cours pour eviter la confusion.

Formater pour afficher

```
maintenant = datetime.now()
print(maintenant.strftime("%Y-%m-%d %H:%M"))
print(maintenant.strftime("%d/%m/%Y"))
```

`strftime` transforme une date en texte selon un motif. `%Y` annee sur 4 chiffres, `%m` mois, `%d` jour, `%H` heure 24h, `%M` minutes. Tu retiendras les motifs en les reutilisant. Pas besoin de les apprendre comme une poesie.

Pour un CSV ou un nom de fichier, le format ISO `2026-07-24` trie bien dans l'ordre alphabetique. Pratique. Chez DanielCraft, on le recommande des qu'on controle le format.

Parser une chaine

Tu lis `"24/07/2026"` dans un fichier. Tu veux un vrai objet date :

```
texte = "24/07/2026"
jour = datetime.strptime(texte, "%d/%m/%Y").date()
print(jour)
```

`strptime` = parse. `strftime` = format. Les deux utilisent le meme langage de motifs. Si le texte ne correspond pas, tu attrapes une `ValueError`. Sam fait planter volontairement avec un mauvais motif. Les eleves retiennent.

Ajouter / retirer des jours

```
aujourd_hui = date.today()
dans_une_semaine = aujourd_hui + timedelta(days=7)
hier = aujourd_hui - timedelta(days=1)
print(dans_une_semaine, hier)
```

`timedelta` sait aussi `hours=`, `minutes=`. Ideal pour "expire dans 48h" ou "fichier plus vieux que 30 jours". Max calcule "dans dix jours" pour ses relances devis. Lea calcule "il y a trente jours" pour purger des logs.

Filtrer des lignes "du mois"

Imagine un CSV avec une colonne `jour` au format `2026-07-24` :

```

from datetime import date
import csv
from pathlib import Path

debut = date(2026, 7, 1)
fin = date(2026, 7, 31)

with Path("data/events.csv").open(encoding="utf-8", newline="") as f:
    for ligne in csv.DictReader(f):
        j = date.fromisoformat(ligne["jour"])
        if debut <= j <= fin:
            print(ligne)

```

`fromisoformat` lit directement `YYYY-MM-DD`. Si tu controles le format de ton CSV, c'est le plus simple.

Timestamp de log

```

import logging
from datetime import datetime

logging.info("Evenement a %s", datetime.now().isoformat(timespec="seconds"))

```

Ou laisse le format `asctime` du logging faire le travail (chapitre precedent). Sans horodatage, un log est une rumeur.

♦ ASTUCE

Pour un CSV ou un nom de fichier, prefere `YYYY-MM-DD`. Ca trie bien, ca se parse avec `fromisoformat`, ca voyage bien.

Petite histoire

Sam avait un CSV d'absences avec des dates en `JJ/MM/AAAA`. Il filtrait "a la main" en triant alphabetiquement. Resultat : septembre avant janvier, chaos total. Avec `strptime` puis comparaison de vrais objets `date`, le filtre du trimestre marche du premier coup. Lea fait pareil pour ses rapports mensuels clients.

Max horodate ses exports factures avec `strftime("%Y-%m-%d")` dans le nom de fichier. Il retrouve "le fichier de mardi" en une seconde. Petit geste, gros confort. Chez DanielCraft, on collectionne ces petits confort : ils restent quand les tutos s'oublient.

Erreur classique

Comparer des chaines `"9/7/2026"` et `"10/7/2026"` en ordre alphabetique... ca ment. Parse en `date`, puis compare. Autre classique : oublier que `strftime` et `strptime` ont besoin du meme motif exact. Autre piege : melanger `date` et `datetime` dans une comparaison sans conversion. Et encore : croire que "maintenant" suffit toujours - parfois tu as besoin d'une date fixe pour un rapport reproductible.

En vrai

Affiche la date du jour en `JJ/MM/AAAA`. Calcule la date dans 10 jours. Parse une chaine de ton choix. Trois gestes, beaucoup d'usages. Puis compare deux dates parsees : sens la difference avec un tri de chaines.

A toi

Ecris `jours_depuis(texte)` qui prend `"2026-01-15"` et retourne le nombre de jours jusqu'à aujourd'hui. Utile pour "ça fait combien de temps ?". Bonus : gère une `ValueError` si le texte est mal formé, avec un message clair. Sam utilise ce genre de fonction pour "combien de jours depuis la dernière absence". Max pour "combien de jours avant la relance devis".

★ A RETENIR

Parse, compare, formate. Des objets date, jamais des chaînes pour l'ordre.

Chapitre 11 - Regex léger avec re



Chercher un motif dans un texte.

Une expression reguliere (**regex**), c'est un motif pour chercher ou valider du texte. Exemple : "est-ce que ca ressemble a un email ?", "trouve tous les numeros", "extraire un code postal". Utile, parfois magique - et souvent dangereuse si tu en abuses.

Une regex, c'est un filtre sur du texte irregulier. Tu decris un motif ("quelque chose qui ressemble a un email"). Python cherche la premiere occurrence, ou toutes, ou verifie que la chaine entiere correspond. Pour du CSV, du JSON, des extensions de fichier : utilise l'outil du format, pas la regex. La regex brille quand le format est sale ou libre.

Attention : les regex peuvent devenir un sport extreme. Ici, on reste léger. Quelques cas utiles avec le module **re**. Chez DanielCraft, consigne : si un **.startswith** ou un **in** suffit, n'utilise pas de regex. Lea extrait des emails dans des exports clients. Max valide des codes postaux dans des adresses. Sam montre le minimum utile sans noyer ses eleves.

★ A RETENIR

Regex oui, mais léger. Si **Path.suffix** ou le module **csv** suffisent, laisse tomber la regex.

search : y a-t-il un motif ?

```
import re

texte = "Contacte-moi : ada@exemple.fr merci"
m = re.search(r"[\w.+]+\@[ \w.-]+\.\w+", texte)
if m:
    print("Email trouve :", m.group())
else:
    print("Pas d'email")
```

Le **r"..."** est une raw string : les backslash restent plus lisibles. **search** trouve la premiere occurrence. **m.group()** donne le texte matche.

Ce motif email est simplifie (pas une validation legale absolue). Pour un exercice ou un filtre grossier, ca va. Pour de la vraie validation metier, les regles sont plus riches.

match vs search vs fullmatch

search : quelque part dans la chaine. **match** : au debut. **fullmatch** : la chaine entiere doit correspondre.

```
print(re.fullmatch(r"\d{5}", "75001")) # code postal FR simplifie
print(re.fullmatch(r"\d{5}", "7500"))  # None
```

Pour valider une saisie complete, `fullmatch` est souvent le bon reflexe.

Groupes

Tu veux extraire des morceaux :

```
m = re.search(r"note\s*[:=]\s*(\d+(?:\.\d+)?)", "note: 14.5")
if m:
    print(float(m.group(1)))
```

Les parentheses capturent. `group(1)` est le premier groupe. Utile pour parser des logs ou des lignes un peu libres.

findall

```
texte = "Alice 14, Bob 11, Chloe 16"
notes = re.findall(r"\d+", texte)
print(notes) # ['14', '11', '16']
```

Tu recuperes une liste de chaines. Convertis ensuite en nombres si besoin.

Cas utiles du quotidien

Verifier qu'un identifiant ne contient que lettres, chiffres, tirets : `r"^[a-zA-Z0-9_-]+$"`. Chercher une URL grossiere : `r"https?:\/\/\S+"`. Remplacer des espaces multiples : `re.sub(r"\s+", " ", texte).strip()`.

```
propre = re.sub(r"\s+", " ", " trop d'espaces ").strip()
print(propre)
```

Quand ne pas regex

Compter des lignes CSV : module `csv`. Parser du JSON : `json`. Verifier une extension de fichier : `Path(p).suffix == ".csv"`. Les regex brillent sur du texte irregulier. Sur des formats structures, utilise l'outil du format.

♦ ASTUCE

Pour valider une saisie complete (code postal, id), prefere `fullmatch`. Pour "trouver quelque part", `search`. Pour "tous les morceaux", `findall`.

Petite histoire

Max voulait extraire des numeros de facture d'un bloc de texte copie depuis un PDF. Il a ecrit une regex de trois lignes illisible, ca marchait une fois sur deux. Lea lui a montre `re.search` avec un groupe simple. Dix lignes, lisible, fiable pour son cas. La lecon : regex oui, mais leger et teste.

Sam montre en cours un email simplifie, puis dit : "ce n'est pas une validation legale". Les eleves comprennent la difference entre filtre grossier et regle metier. Chez DanielCraft, on prefere un filtre honnete a une regex "parfaite" de quatre-vingts caracteres.

Erreur classique

Ecrire une regex illegible de trois lignes pour un probleme soluble en deux `split`. Ou oublier `r"..."` et se battre avec les `\` escapes. Ou croire qu'une regex email "parfaite" existe en dix caracteres. Ou utiliser regex la ou `Path.suffix` suffit.

Zoom : lire un log avec regex

Parfois un CSV ou un JSON n'existe pas. Tu as une ligne de log libre :

```
2026-07-24 ERROR user=alice note=14.5 action=save
```

Tu peux extraire la note sans tout parser :

```
m = re.search(r"note=(\d+(?:\.\d+)?)", ligne)
if m:
    print(float(m.group(1)))
```

C'est le bon usage : texte irregulier, extraction cillee. Si le format devient stable, passe a un vrai parseur (CSV, JSON, config). Lea utilise ce genre de regex une fois par mois sur des exports clients brouillons. Max prefere `csv` des que le format se stabilise.

En vrai

Prends un paragraphe avec un email et un numero a 5 chiffres. Extraie les deux avec `search` / `findall`. Puis valide une saisie code postal avec `fullmatch`.

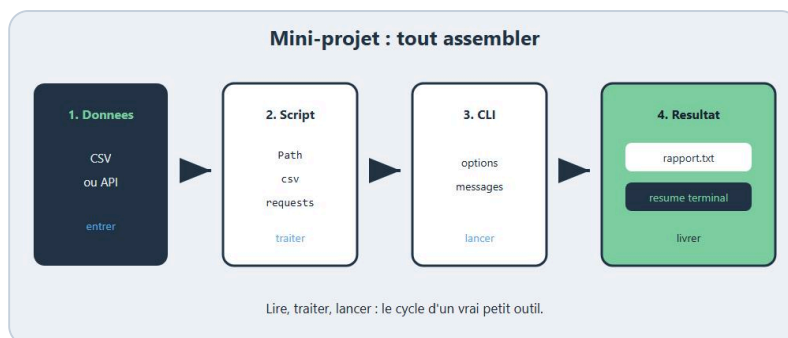
A toi

Ecris `extraire_emails(texte)` qui retourne une liste (peut-etre vide). Teste avec 0, 1, 2 emails dans la chaine. Simple, utile, et tu as touche le coeur de `re` sans te noyer. Si tu sens l'envie d'une regex de trois lignes, arrete-toi et demande-toi si un `split` ou le module `csv` suffirait. Chez DanielCraft, cette pause evite plus de dettes que n'importe quel motif "elegant".

⚠ ATTENTION

Une regex illisible de trois lignes pour un `split` suffit rarement. Prefere le plus simple qui marche, puis teste.

Chapitre 12 - Mini-projet : CLI resume CSV ou API



Mini-projet : CLI + donnees + resume clair.

On assemble. Objectif : un petit outil en ligne de commande qui, selon le mode, lit un **CSV** de notes ou appelle une **API** meteo, puis affiche un resume clair. Avec arguments, gestion d'erreurs, et un log discret. C'est le moment ou la theorie devient un outil sous le coude.

Tu n'as pas besoin que ce soit parfait. Il faut que les deux chemins marchent, et que l'echec soit humain. Un seul script, deux modes. Mode notes : tu passes un fichier CSV et un nom d'eleve, tu obtiens une moyenne. Mode meteo : tu passes des coordonnees (ou des defaults Paris), tu obtiens une temperature. Les deux modes partagent la meme structure : argparse, fonctions metier, print pour l'utilisateur, logging pour le detail.

C'est le moment ou Lea, Max et Sam retrouvent leurs trois fils rouges : notes CSV, meteo API, script **CLI**. Chez DanielCraft, c'est ce genre de petit outil qu'on garde parce qu'il sert vraiment.

★ A RETENIR

Range ton fichier dans cet ordre : imports, fonctions metier, fonctions CLI, puis `main`. Tu te retrouves plus vite quand ca casse.

Parcours

1. 1. Cree un dossier de projet avec un venv et `requests` installe.
2. 2. Place un `data/notes.csv` d'exemple.
3. 3. Ecris `resume.py` avec argparse : `--mode notes|meteo`, et les options utiles.
4. 4. Mode notes : `--fichier + --eleve -> moyenne`.
5. 5. Mode meteo : latitude/longitude (ou ville fixe) -> temperature.
6. 6. Erreurs : fichier manquant, eleve inconnu, HTTP rate, timeout.
7. 7. `print` pour l'utilisateur, `logging` pour le detail.

Squelette argparse

```
import argparse
import logging

def build_parser():
    p = argparse.ArgumentParser(description="Resume notes CSV ou meteo API")
    p.add_argument("--mode", choices=["notes", "meteo"], required=True)
    p.add_argument("--fichier", help="CSV pour le mode notes")
    p.add_argument("--eleve", help="Nom de l'eleve")
    p.add_argument("--lat", type=float, default=48.85)
    p.add_argument("--lon", type=float, default=2.35)
    return p
```

Tu branches ensuite :

```
def main():
    logging.basicConfig(level=logging.INFO, format="%(levelname)s - %(message)s")
    args = build_parser().parse_args()
    if args.mode == "notes":
        commande_notes(args)
    else:
        commande_meteo(args)

if __name__ == "__main__":
    main()
```

Le `if __name__ == "__main__"` evite d'executer `main` si le fichier est importe. Habitude saine.

Mode notes (coeur)

Reutilise `pathlib` + `csv.DictReader`. Calcule la moyenne. Si aucune ligne : message clair. Si le fichier n'existe pas : "Fichier introuvable : ..." et un `logging.error`. Tu as deja ecrit ces gestes dans les chapitres CSV et CLI. Ici, tu les branches derriere `--mode notes`. Sam reconnait son atelier. Lea reconnait son export client. Max reconnait ses factures sous d'autres colonnes.

Idee de coeur (a adapter) :

```
def moyenne_eleve(chemin, eleve):
    total = 0.0
    compte = 0
    with Path(chemin).open(encoding="utf-8", newline="") as f:
        for ligne in csv.DictReader(f):
            if ligne["eleve"] == eleve:
                total += float(ligne["note"])
                compte += 1
    if compte == 0:
        return None
    return total / compte
```

Mode meteo (coeur)

`requests.get` avec timeout, `raise_for_status`, extraction de la temperature. En cas d'echec : "Impossible de recuperer la meteo." + log de l'exception. Meme discipline que l'atelier API : params separees, timeout explicite, message humain. Max lance ce mode le matin. Lea ajoute parfois `--brut` pour debug. Toi, tu verifies d'abord le chemin malheureux (timeout court) avant de dire "c'est fini".

Qualite minimale

Pas de traceback brut comme seul resultat. Pas de secret en dur. Aide `-h` lisible. Noms de fonctions clairs (`commande_notes`, `moyenne_eleve`, `fetch_meteo`). Un resume en une ou deux lignes a l'ecran.

Exemple de sorties attendues :

```
Moyenne de Alice : 15.0 (4 notes)
Il fait 18.2 C (lat=48.85, lon=2.35)
```

Organisation minimale du fichier

Meme dans un seul `resume.py`, range dans cet ordre : imports, logging config dans `main`, fonctions metier (`moyenne_eleve`, `fetch_meteo`), fonctions CLI (`commande_notes`, `commande_meteo`), puis `main`. Tu te retrouves plus vite. Plus tard, tu pourras couper en plusieurs fichiers (chapitre organisation).

Petite histoire

Sam avait lu tous les chapitres mais n'avait rien assemble. Lea lui a dit : "Fais le mini-projet ce week-end, meme moche." Sam l'a fait en deux heures. Lundi, il calculait les moyennes du trimestre en une commande. Max a ajoute le mode meteo pour son usage perso. Deux modes, un script, zero copier-coller.

Lea a garde ce squelette comme modele pour ses outils clients. Chez DanielCraft, on dit souvent : un mini-projet fini bat dix chapitres survoles.

Variante

Ajoute `--seuil` en mode notes pour lister les notes au-dessus du seuil. Ou `--json` pour afficher un mini objet JSON au lieu d'une phrase. Bonus, pas obligatoire.

Criteres "c'est fini"

Tu peux expliquer en trente secondes comment lancer les deux modes. Un ami peut lire `-h` et comprendre. Tu as casse au moins deux choses volontairement (mauvais fichier, mauvais reseau) et les messages tenaient la route. Le code n'a pas de cle en dur.

⚠ ATTENTION

Casse volontairement le CSV et le reseau avant de dire "c'est fini". Les chemins malheureux comptent autant que le chemin heureux.

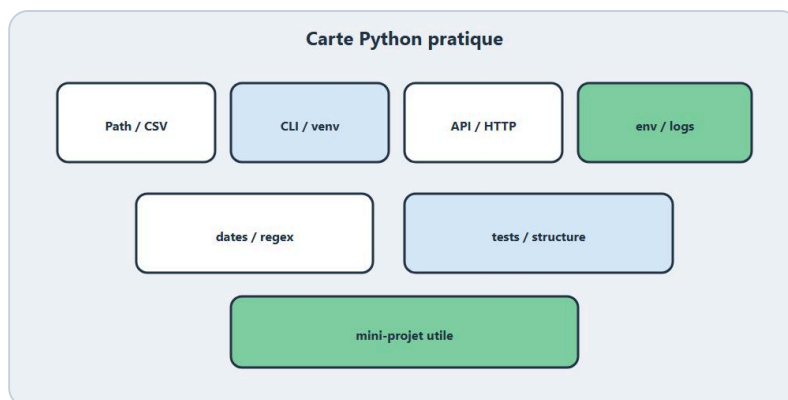
A toi

Code les deux modes. Casse volontairement le chemin CSV. Casse l'URL ou coupe le reseau. Verifie les messages. Remets tout. Si les chemins heureux et malheureux sont propres, le mini-projet est reussi.

♦ **ASTUCE**

Deux modes, un CLI, messages humains, secrets dehors. Assemble, casse, reparer : c'est le vrai test.

Chapitre 13 - A retenir (carte mentale)



Carte Python pratique.

Si tu ne devais garder qu'une page, ce serait celle-ci. Python pratique, ce n'est pas "savoir coder des exercices". C'est savoir lire un **tableau**, lancer un script avec des **options**, appeler une **API**, et ranger un petit projet sans paniquer. Tu pilotes. Python execute. Tu verifies. Tu assumes. Chez DanielCraft, on colle cette carte au-dessus de l'écran quand un script devient flou. Relis-la avant les ateliers. Relis-la après. Elle sert de boussole.

Tu n'as pas besoin de tout memoriser. Tu as besoin de reconnaître les gestes et de savoir où les retrouver. Lea resume en une phrase : "fichier ou API, message clair, secret dehors". Max dit : "venv, timeout, **-h**". Sam dit : "je peux relancer demain sans me souvenir de tout". Les trois ont raison. Choisis ta phrase. Garde-la.

Trois fils rouges sur une même table. Un CSV de notes (Sam). Une météo JSON (Max). Un script CLI avec **-h** (Lea). Autour : pathlib, logging, secrets, dates, regex avec moderation. Le mini-projet assemble. Les ateliers forcent le livrable. Les tests protègent le cœur. L'organisation évite le dossier "Nouveau dossier (3)". Tout le livre tient dans ces gestes.

Ce que ce n'est pas

Ce chapitre n'est pas un glossaire de soixante mots. Ce n'est pas non plus un examen surprise. Ce n'est pas "la fin de Python". Et ce n'est surtout pas le moment de paniquer parce qu'un paramètre `argparse` t'échappe. Les gestes comptent plus que le catalogue. Si tu peux ouvrir un CSV, lancer un CLI, faire un GET avec timeout, et isoler un venv, tu as le socle.

★ A RETENIR

"Fichier ou API, message clair, secret dehors, venv isole." Cette phrase resume le livre mieux qu'un glossaire.

Les idées solides

`pathlib` : chemins en objets, `/` pour assembler, `read_text` / `write_text`, ancrer avec `Path(__file__).parent`. `csv` : `DictReader` / `DictWriter`, `newline=""`, `encoding="utf-8"`, tout est string jusqu'à conversion. `argparse` : options CLI, aide `-h`, `required=True` dès le début. `venv` + `pip` : un projet, un venv, `python -m pip install -r requirements.txt`. `requests` : GET, `.json()`, `params`,

`timeout`, `raise_for_status`. Secrets : `os.environ`, `.env` hors Git, `.env.example` partageable. `logging` : niveaux, `print` pour l'utilisateur, `log` pour le detail. `datetime` : `strftime`, `strptime`, `timedelta`, comparer des objets pas des chaines. `re` : `search`, `fullmatch`, `findall` avec moderation.

Ce n'est pas une checklist a cocher en courant. C'est un paysage. Si un mot est flou, rouvre le chapitre concerne dix minutes. Une relecture active bat une lecture passive de quarante pages.

La boucle DanielCraft

1) Lire ou appeler. 2) Verifier (fichier existe, HTTP ok, JSON attendu). 3) Traiter (calcul, filtre, resume). 4) Afficher clair (`print`). 5) Tracer le detail (`logging`). 6) Echouer proprement (message humain, pas traceback brut). 7) Ranger (venv, requirements, README).

C'est la meme boucle pour Lea sur un export client, pour Max sur la meteo du chantier, pour Sam sur les notes du trimestre. Les donnees changent. Le geste reste.

Ce que tu peux oublier

La syntaxe exacte de chaque parametre `argparse`. Le nom de chaque exception `requests`. La peur du terminal. Garde les gestes. Change de paquet si besoin. Les gestes restent. Tu n'es pas un dictionnaire ambulant. Tu es un pilote qui sait ou chercher.

Petite checklist de poche

Venv actif ? Timeout sur le reseau ? Secrets hors du code ? CSV avec `newline=""` ? Chemins ancrees si besoin ? `-h` lisible ? Un test minimal sur le coeur du calcul ? Si oui partout, tu es pret pour les ateliers.

♦ ASTUCE

Sans regarder tes notes, ecris de memoire : ouvrir un CSV avec `DictReader`, et un `requests.get` avec `timeout` + `raise_for_status`. Les trous montrent ce qu'il faut relire.

Petite histoire

Sam avait lu tous les chapitres. Le lundi des notes, il a bloque sur "par quel bout commencer". Lea lui a dit : "Relis la carte. Fais le CSV. Puis le CLI. Puis l'API." En deux soirees, le mini-projet etait la. Max a saute la carte, a melange secrets et chemins absolus, a perdu une heure. Il est revenu a cette page. Chez DanielCraft, on voit souvent cette scene : la carte poche vaut mieux que la memoire heroique.

Erreur classique

Croire que "je connais les bases" egal "je sais automatiser mon quotidien". Les bases sont le moteur. La pratique, c'est la boite a outils. Autre piege : empiler `pandas`, `Flask` et `Docker` avant d'avoir un script CSV + CLI + `timeout` qui tient la route. Autre piege : lire cette carte sans refaire un "A toi" a voix haute.

⚠ ATTENTION

Sans livrable apres la carte, tu "comprends" et tu oublies. Ecris. Lance. Casse. Repare.

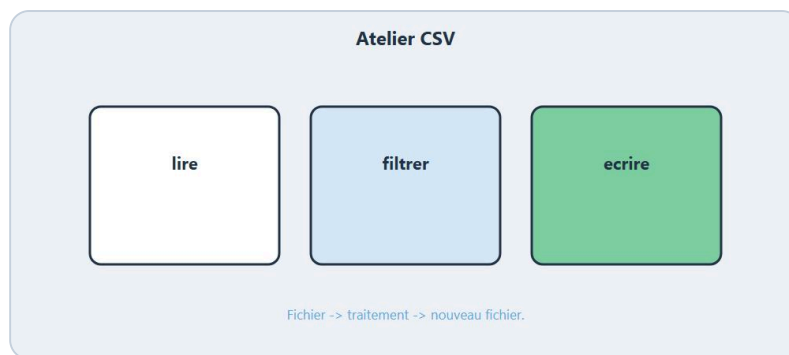
En vrai

Sans regarder le livre, écris sur papier en dix lignes : pathlib, csv, argparse, venv, requests, secrets, logging. Compare ensuite. Les trous montrent ce qu'il faut relire avant les ateliers. Pas de honte. De la carte.

A toi

Recopie cette carte sur papier en 10 lignes max, avec TES mots. Si tu peux l'expliquer à un ami sans regarder, c'est bon signe. Prends le temps. Un atelier fait à fond vaut mieux que trois ateliers survolés. Si tu es presse, fais la moitié aujourd'hui et l'autre demain - mais écris le livrable. Sans livrable, le cerveau classe ça comme "lu", pas comme "su". DanielCraft forme des gens qui livrent, même petit.

Chapitre 14 - Atelier : CSV des notes



Lire, filtrer, ecrire.

Objectif : un petit pipeline local. Tu crées un **CSV**, tu calcules des **moyennes** par eleve, tu écris un second CSV resume. Pas d'API ici. Que des fichiers et du Python clair. Durée : 30 a 45 minutes. Matériel : Python 3, un éditeur, un terminal.

Sam veut remplacer son Excel de notes par un script réutilisable. Lea lui a donné cet atelier comme premier pas concret. Max fait la variante avec ses factures fournisseurs (même logique, autres colonnes). Chez DanielCraft, on considère qu'un atelier sans livrable, c'est une lecture déguisée. Tu vas sortir avec un dossier **atelier-csv/** que tu peux rouvrir dans six mois sans paniquer.

Entrée : **notes.csv** avec des lignes eleve / matiere / note. Sortie : **moyennes.csv** avec eleve / moyenne / nb_notes. Au milieu : **DictReader**, conversion **float**, accumulation, **DictWriter**. Si le fichier manque, message clair. Si une note est pourrie, tu décides : ignorer ou arrêter. Le script se relance deux fois de suite sans planter. C'est ça, un outil, pas un exercice jetable.

★ A RETENIR

Avant de coder, écris à la main une moyenne d'un eleve sur ton CSV. Tu auras une vérité à comparer après le script.

Exercice 1 - Préparer les données (10 min)

Crée le dossier **atelier-csv** avec un sous-dossier **data**. Écris **data/notes.csv** avec l'en-tête **eleve,matiere,note** et au moins 6 lignes (2 eleves, 3 matieres). Utilise **pathlib** pour vérifier que le dossier existe. Encoding UTF-8. Évite les accents dans les chemins si ton terminal est capricieux ; dans les données, UTF-8 doit tenir.

Exercice 2 - Lire et calculer (15 min)

Écris **resume_notes.py** qui utilise **pathlib** et **csv**. Lis toutes les notes avec **DictReader**. Accumule somme et compte par eleve (dictionnaire de dictionnaires, ou deux dicos). Affiche dans le terminal chaque eleve et sa moyenne arrondie à 1 decimale. Convertis les notes avec **float**. Souviens-toi : tout est string jusqu'à conversion.

Exercice 3 - Ecrire le resume (10 min)

Ecris `data/moyennes.csv` avec les colonnes `eleve,moyenne,nb_notes`. Utilise `DictWriter` avec `writeheader()`. Relance le script deux fois de suite : le fichier doit etre recree sans planter. C'est le test "outil du quotidien" : Sam le relancera chaque trimestre.

Exercice 4 - Gerer l'echec (5 min)

Gere le cas fichier manquant avec un message clair (pas seulement le traceback). Si une note est invalide, decide : ignorer avec un warning, ou arreter avec une erreur claire. Documente ton choix en commentaire. Lea prefere arreter tot sur un export client. Max prefere ignorer une ligne foireuse et logger. Choisis, mais choisis.

Idee de structure interne

```
def lire_notes(chemin):  
    ...  
  
def moyennes_par_eleve(lignes):  
    ...  
  
def ecrire_moyennes(chemin, moyennes):  
    ...
```

Tu testes mentalement chaque morceau. Au chapitre tests, tu pourras poser des `assert` sur `moyennes_par_eleve` sans toucher aux fichiers. Separe le coeur du CLI des maintenant : demain tu te remercieras.

Petite histoire

Sam a calcule les moyennes a la main sur Alice pendant que Lea ecrivait le script. Quand le script a dit 14.5 et le papier 14.5, Sam a sourit. Max a copie le meme schema pour ses factures : fournisseur, montant, total. Meme pipeline, autre metier. Chez DanielCraft, on aime ces ponts : un atelier notes qui devient un outil perso en une soiree.

Le premier echec de Sam ? Notes laisseees en string. `"10" + "12"` ne fait pas 22. La conversion `float` a tout debloque. Tu vas peut-etre vivre la meme scene. Tant mieux : tu ne l'oublieras plus.

Livable

Un dossier `atelier-csv/` avec `data/notes.csv`, `data/moyennes.csv`, et `resume_notes.py`. Les moyennes verifiees a la main sur au moins un eleve.

Criteres de reussite

Relancer le script deux fois de suite recree `moyennes.csv` sans planter. Les moyennes sont correctes (verifie a la main sur un eleve). Le code utilise `DictReader` / `DictWriter`, pas un `.split(",")` maison. Encoding UTF-8 partout.

Bonus

Ajoute argparse : `--fichier` pour le CSV source, `--sortie` pour le CSV moyennes. Valeurs par default vers `data/notes.csv` et `data/moyennes.csv`. Si tu fais le bonus, tu anticipes l'atelier CLI.

Erreur classique

Les notes en string oubliées. Oublier `writeheader` et te retrouver avec un CSV sans colonnes nommées. Utiliser `.split(",")` sur un CSV qui contient des virgules dans une cellule. Autre piège : travailler dans le mauvais dossier et croire que le fichier "n'existe pas".

⚠ ATTENTION

Tout est string jusqu'à `float`. Oublier la conversion, c'est le piège numero un de cet atelier.

En vrai

Ouvre `moyennes.csv` dans un tableur. Vérifie une ligne à la main. Si ça matche, tu as un pont Python <-> tableur. C'est souvent ça le vrai gain pour Sam, Lea et Max.

A toi

Quand `moyennes.csv` est correct, écris trois lignes : ce qui t'a pris le plus de temps, ce que tu referais différemment, ce que tu gardes tel quel. Garde ce papier. L'atelier suivant partira de ce réflexe "outil avec options". Si tu as ignoré une note invalide, écris aussi pourquoi : demain tu sauras ce que tu as choisi.

♦ ASTUCE

Un atelier fait à fond bat trois ateliers survolés. Écris le livrable aujourd'hui, même si tu finis demain.

Chapitre 15 - Atelier : CLI utile



argparse : options au lancement.

Objectif : transformer un script "en dur" en outil avec **arguments**. Tu dois pouvoir changer de fichier et d'élève sans éditer le code. Durée : 30 à 45 minutes. Matériel : ton script CSV ou équivalent, Python 3, terminal.

Max avait un script qui marchait... chez lui, avec son fichier, pour Alice. Lea lui a dit : "Ajoute **argparse**, et montre-moi **-h**." C'est cet atelier. Sam en profite pour rendre son outil de notes utilisable par un collègue sans formation Python. Chez DanielCraft, un script sans **-h** lisible, c'est un brouillon, pas un outil. Tu vas sentir la différence entre "ça marche si je connais le code" et "ça marche si je lis l'aide".

Avant : ouvrir l'éditeur, changer "Alice" en "Bob", sauver, lancer. Après : `python moyenne_cli.py --fichier data/notes.csv --eleve Bob`. Même calcul. Autre posture. Les options obligatoires refusent tout. Les options optionnelles enrichissent. Les échecs (fichier absent, élève inconnu) parlent français. C'est le passage "script perso" vers "outil partageable".

♦ ASTUCE

Montre **-h** à quelqu'un (ou à toi demain). Si on comprend sans lire la source, l'atelier est déjà à moitié gagné.

Exercice 1 - Arguments de base (10 min)

Reprends (ou recrée) un script qui calcule la moyenne d'un élève depuis un CSV. Ajoute **argparse** avec **-fichier** (obligatoire) et **--eleve** (obligatoire). Vérifie que `python script.py -h` affiche une aide compréhensible en français. Si l'aide est en jargon, réécris les `help=`.

Exercice 2 - Options avancées (10 min)

Ajoute **--seuil** optionnel (float). Si présent, affiche aussi combien de notes sont $\geq$ seuil pour cet élève. Ajoute **--verbose** (`store_true`). En verbose, logue chaque ligne retenue (via **logging** ou **print** clairement marqué). Lea adore **--verbose** quand un client dit "c'est faux" : elle voit quelles lignes ont été prises.

Exercice 3 - Cas d'echec (10 min)

Teste au moins trois lancements : eleve connu, eleve inconnu, fichier inexistant. Eleve inconnu -> message du type "Aucune note pour X", code retour non zero possible (`sys.exit(1)`). Fichier manquant -> message clair. Aucune valeur secrete, aucun chemin absolu grave dans le code. Si argparse refuse une option manquante, c'est une victoire, pas un echec.

Exercice 4 - Montrer a quelqu'un (5 min)

Montre a quelqu'un (ou a toi-meme demain) uniquement l'aide `-h` et un exemple d'appel. Si on comprend sans lire le source, l'atelier est reussi. Sam fait souvent cet exercice avec un collegue non tech : "lance ca pour Alice".

Exemple d'appel

```
python moyenne_cli.py --fichier data/notes.csv --eleve Alice --seuil 12 --verbose
```

Petite histoire

Max a montre son `-h` a Lea. Lea a dit : "Je comprends." Max a sourit. Deux jours plus tard, il a change de CSV fournisseur sans toucher une ligne de code. Sam a ajoute `--seuil` pour reperer les eleves sous la moyenne. Chez DanielCraft, on mesure le succes d'un CLI a une question : demain matin, fatigue, est-ce que tu retrouves comment l'appeler sans rouvrir le source ?

Livable

Un script `moyenne_cli.py` (ou nom equivalent) avec argparse complet, trois tests manuels documentes (eleve connu, inconnu, fichier absent), et une capture ou note de l'aide `-h`.

Criteres de reussite

L'aide `-h` explique chaque option sans jargon inutile. Eleve inconnu et fichier manquant produisent des messages clairs. Aucun secret, aucun chemin absolu en dur. Si tu coches ces trois points, tu as un outil partageable, pas seulement un script perso.

Bonus

Accepte un argument positionnel `fichier` a la place de `--fichier`, et garde `--eleve`. Habitue-toi aux deux styles. Ou ajoute `--sortie` pour ecrire aussi un mini CSV resume. Lea ajoute souvent `--sortie` sur ses exports clients. Max prefere rester simple : terminal seulement. Sam demande les deux styles pour que les eleves voient la difference.

Erreur classique

Tout valider "plus loin" dans le code alors qu'argparse peut deja exiger les options. Si `--eleve` manque, mieux vaut le refus immediat d'argparse qu'un `None` mysterieux au milieu du calcul. Autre piege : laisser un chemin `C:\Users\...` en dur "juste pour tester" et l'oublier. Autre piege : une aide `-h` en anglais cryptique alors que ton public lit le francais.

⚠ ATTENTION

`required=True` dès le début bat un `None` mystérieux au milieu du calcul. Laisse `argparse` refuser tout.

En vrai

Lance ton script avec `-h`. Copie l'aide dans un fichier `AIDE.txt` ou en commentaire en haut du script. Relance demain sans ouvrir l'éditeur : est-ce que tu retrouves les options ?

A toi

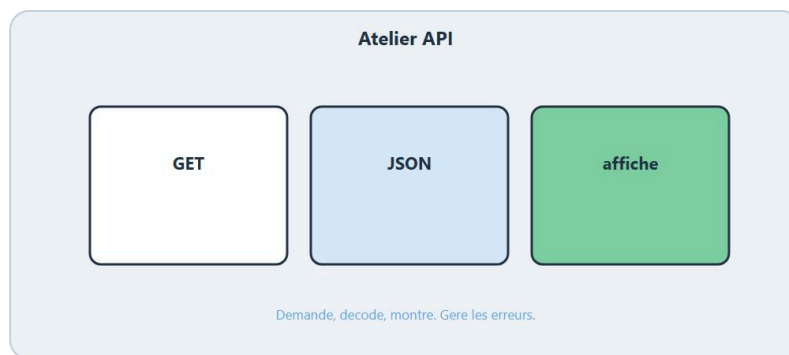
Documente en haut du fichier deux exemples d'appel commentés. Ton futur toi fatigué te remerciera. C'est l'esprit des petits outils DanielCraft : utilisables, pas seulement "techniquement corrects".

Quand tu bloques sur une option, relis l'aide `-h` à voix haute. Si tu hésites, un collègue hésite aussi. Lea passe souvent cinq minutes à réécrire les `help=` avant de livrer. Max a appris à forcer trois lancements (ok, inconnu, fichier absent) avant de dire "c'est fini". Sam note ces trois lancements dans un commentaire en haut du fichier. Chez DanielCraft, ce rituel suffit à transformer un script perso en outil partageable.

★ A RETENIR

Un CLI utile : options claires, `-h` lisible, échecs propres, zéro chemin en dur.

Chapitre 16 - Atelier : appel API meteo



GET, JSON, affichage, erreurs.

Objectif : un script qui recupere la meteo actuelle via une **API** JSON publique, affiche un resume, et gere proprement les echecs reseau. Travaille dans un **venv** avec **requests**. Duree : 30 a 45 minutes.

Max check la meteo chaque matin avant les chantiers extérieurs. Lea veut un outil reutilisable, pas un copier-coller d'URL. Sam montre a ses eleves comment le reseau entre dans Python. Chez DanielCraft, un script API sans **timeout** ni gestion d'erreur, c'est un demo, pas un outil. Tu vas sentir la difference entre "ca a marche une fois" et "ca explique quand ca casse".

Tu lances `python meteo.py --lat 48.85 --lon 2.35`. Le script fait un GET avec params et timeout, verifie le statut, extrait la temperature, affiche une phrase claire. Si le reseau rale, message humain + log. Si une cle JSON manque, tu ne pretend pas que tout va bien. Meme schema que le CSV : lire, verifier, traiter, afficher, echouer proprement. Toi, tu actives le venv pour de vrai avant de lancer.

★ A RETENIR

Teste un timeout tres court (0.001) des l'exercice 4. Si le message est propre, tu as gagne le reflexe "echec reseau".

Exercice 1 - Environnement (5 min)

Cree un dossier **atelier-api** et un venv. Installe **requests**. Ecris un **requirements.txt**. Active le venv avant chaque session. Si tu installes globalement "pour aller plus vite", tu te prepares une galere sur une autre machine.

Exercice 2 - Appel de base (10 min)

Choisis une API simple (Open-Meteo convient : pas de cle pour un usage de base). Ecris **meteo.py** avec argparse : `--lat`, `--lon` (defaults Paris ok), eventuellement `--timeout`. Fais un GET avec **params** et **timeout**. Pas d'URL monstrueuse colle en dur sans params : les parametres se lisent mieux separes.

Exercice 3 - Extraction et affichage (10 min)

Verifie le succes (`raise_for_status` ou test du status). Extrais temperature (et vent si dispo). Affiche une phrase claire du type "Temperature : 18 C, vent 12 km/h". **print** pour l'humain. **logging** pour le detail si tu veux pousser.

Exercice 4 - Echechs propres (10 min)

Attrape timeout et erreurs reseau : message utilisateur + log. Un timeout tres court (ex: 0.001) doit produire un message propre, pas seulement un mur de traceback. Aucune cle en dur dans le fichier (meme si l'API n'en demande pas : prends le reflexe). Lea teste toujours le chemin malheureux avant de montrer un outil a un client.

Exercice 5 - Variable d'environnement (5 min)

Ajoute une variable d'environnement optionnelle `METEO_TIMEOUT` pour surcharger le timeout si presente. Documente-la dans un `.env.example`. Meme sans vrai secret ici, tu ancrages l'habitude "config dehors".

Squelette d'extraction

```
data = reponse.json()
meteo = data["current_weather"]
temp = meteo["temperature"]
print(f"Temperature : {temp} C")
```

Adapte les clefs si tu changes d'API. L'important est le parcours, pas la marque du service. Protege les `KeyError` : un 200 ne garantit pas la forme que tu aimes.

Petite histoire

Max a lance son script sur le parking avant une toiture. Timeout court, message clair, il a regarde le ciel et decide. Lea a ajoute `--brut` pour debug quand Open-Meteo a change une cle. Sam a fait planter le timeout volontairement en classe : les eleves ont vu qu'un echec propre fait partie du metier. Chez DanielCraft, on repete : le chemin malheureux compte autant que le chemin heureux.

Livrable

Un dossier `atelier-api/` avec `meteo.py`, `requirements.txt`, `.env.example`, et une note de trois tests (appel normal, timeout court, lat/lon differentes).

Criteres de reussite

`python meteo.py -h` est lisible. Un appel normal affiche temperature (et unite). Un timeout tres court produit un message propre. Aucune cle en dur.

Bonus

Mode `--brut` qui affiche le JSON indente (`json.dumps(data, indent=2)`) pour debug. Ou ecriture d'une ligne dans `data/historique.csv` avec date + temperature (`datetime` + `csv`). Tu relies alors trois chapitres en un geste.

Erreur classique

Oublier que la structure JSON peut changer. Protege les `KeyError`. Ne suppose pas que "200" implique les clefs que tu aimes. Oublier le timeout. Installer `requests` hors venv et ne plus savoir recreer l'environnement. Autre piege : coller une cle API "temporaire" dans le fichier alors que tu n'en as meme pas besoin ici.

⚠ ATTENTION

Status 200 ne garantit pas les clés que tu attends. Protège les `KeyError`. Et mets toujours un timeout.

En vrai

Lance le script pour deux paires lat/lon différentes. Compare. Puis force un timeout absurde. Si les trois comportements sont clairs, tu as un outil, pas un brouillon.

A toi

Si tu sens que c'est devenu un outil (et plus un exercice), écris en trois lignes : comment tu l'appelles le matin, ce que tu fais si ça rate, ou vit le timeout. Garde ce papier à côté du mini-projet. Relance le script demain sans rouvrir le chapitre : si tu retrouves l'appel et le message d'échec, l'atelier a tenu.

Quand tu bloques sur une clé JSON, affiche d'abord `list(data.keys())` (ou le mode `--brut`) avant de chercher au hasard. Lea fait ça systématiquement. Max aussi, depuis qu'il a passé vingt minutes à chercher `temp` au lieu de `temperature`. Sam montre ce geste en classe : "interroge le JSON avant de l'inventer". Chez DanielCraft, ce réflexe évite plus de paniques que n'importe quel tutoriel d'API.

♦ ASTUCE

GET, timeout, `raise_for_status`, message humain, secrets dehors. Un atelier API propre, c'est ça.

Chapitre 17 - Tests simples



assert : proteger contre les regressions.

Quand une fonction calcule une moyenne ou parse une date, tu veux pouvoir verifier sans relancer tout le CLI a la main. Les **tests**, c'est ca : des mini-verifications automatiques. Pas un luxe de grande entreprise. Un filet pour ton futur toi fatigue, qui modifiera une ligne et cassera un cas limite sans le voir.

On reste simple. D'abord **assert**. Ensuite l'idée de **pytest**. Pas besoin d'une usine. Separe le coeur (fonctions pures : entree -> sortie) du CLI (argparse, print, fichiers). Teste le coeur avec des donnees fictives en memoire. Le CLI, tu le testes manuellement ou plus tard avec des tests d'integration. Commence petit : trois asserts sur une fonction, c'est deja enorme.

Lea teste **moyenne_eleve** avec des dicts en memoire avant de brancher le CSV. Max verifie son extracteur JSON meteo avec un faux dict. Sam montre que trois asserts valent mieux qu'un relancement manuel a chaque modification. Chez DanielCraft, on dit : teste le coeur, pas le theatre du terminal.

Ce que ce n'est pas

Tester, ce n'est pas ecrire plus de code que le programme lui-meme. Ce n'est pas non plus "couvrir 100% des lignes" des le jour un. Ce n'est pas une religion. Trois asserts utiles battent cinquante tests vides. Et ce n'est surtout pas une excuse pour ne plus jamais lancer le script a la main : les tests etent le coeur, tes doigts verifient le CLI.

★ A RETENIR

Fonction pure + asserts = filet. CLI a part. Reseau fictif d'abord, vrai reseau ensuite.

assert : la base

```
def moyenne(valeurs):
    if not valeurs:
        return None
    return sum(valeurs) / len(valeurs)

assert moyenne([10, 20]) == 15
assert moyenne([]) is None
print("Tests OK")
```

Si un `assert` echoue, Python leve `AssertionError` et s'arrete. Si tout passe, tu vois "Tests OK". Pour de petites fonctions pures (entree -> sortie, sans reseau), c'est deja enorme. Max a commence comme ca, sans installer quoi que ce soit. Sam aussi, en classe, sans wifi.

Separer le coeur du CLI

Le piege : tout coller dans `main` avec `argparse`. Difficile a tester. Mieux :

```
def moyenne_eleve(lignes, nom):
    notes = [float(l["note"]) for l in lignes if l["eleve"] == nom]
    if not notes:
        return None
    return sum(notes) / len(notes)
```

Tu testes `moyenne_eleve` avec une liste de dicts en memoire. Pas besoin de fichier. Le CLI, lui, lit le CSV puis appelle la fonction. Lea a decouvert ce decoupage en voulant tester sans toucher au vrai CSV client. Depuis, elle ne revient plus en arriere.

L'idee pytest

`pytest` est un outil populaire. Tu installes dans le venv : `pip install pytest`. Tu ecris un fichier `test_moyenne.py` :

```
from resume import moyenne_eleve

def test_moyenne_simple():
    lignes = [
        {"eleve": "Alice", "note": "10"},
        {"eleve": "Alice", "note": "20"},
    ]
    assert moyenne_eleve(lignes, "Alice") == 15

def test_eleve_absent():
    assert moyenne_eleve([], "Bob") is None
```

Puis tu lances :

```
pytest
```

`pytest` trouve les fonctions `test_...`, les execute, et resume les succes/echec. Plus confortable que des `assert` disperses quand le projet grandit.

Tu n'es pas obligé d'installer `pytest` aujourd'hui. Comprends surtout : fonctions testables + verifications automatiques. L'outil suit l'habitude, pas l'inverse.

Quoi tester en priorite ?

Les calculs. Les parsers (date, validation regex legere). Les branches "vide / absent". Moins urgent au debut : tout le reseau (ca demande des mocks). Pour l'API, tu peux tester une fonction qui extrait la temperature d'un dict JSON fictif.

```
def temperature_depuis(data):  
    return float(data["current_weather"]["temperature"])  
  
assert temperature_depuis({"current_weather": {"temperature": 18.2}}) == 18.2
```

♦ ASTUCE

Fais echouer un assert volontairement une fois. Lis le message. Puis reparer. Tu retiendras mieux qu'avec une lecture passive.

Petite histoire

Sam a modifié son calcul de moyenne et a cassé le cas "élève absent" sans s'en rendre compte. Lea lui a fait écrire trois asserts en cinq minutes. Désormais, Sam lance `checks.py` avant chaque session de notes. Un assert rouge vaut dix relancements manuels.

Max a cassé son extracteur météo en "simplifiant" une clé JSON. Son assert a crié. Il a souri : mieux vaut un assert rouge qu'un mail client avec `KeyError`. Chez DanielCraft, on collectionne ces petites peurs évitées.

Erreur classique

Tester uniquement le chemin heureux. Ajoute le cas vide, le cas absent, une note non numérique si tu la gères. Autre classique : asserts sur le texte exact d'un message utilisateur qui change souvent - préfère tester la valeur de retour. Autre piège : tester le réseau directement (lent, fragile) au lieu de tester l'extraction sur un dict fictif.

Fichier `checks.py` minimal

Tu peux regrouper tes vérifications dans un fichier lancable :

```
# checks.py  
from notes_lib import moyenne_eleve  
  
lignes = [{"eleve": "Alice", "note": "10"}, {"eleve": "Alice", "note": "20"}]  
assert moyenne_eleve(lignes, "Alice") == 15  
assert moyenne_eleve(lignes, "Bob") is None  
print("Tous les checks OK")
```

Lance avec `python checks.py`. Zero dépendance extra. Quand le projet grandit, migre vers `pytest`. L'important : le réflexe "je modifie une fonction, je relance les checks" avant de toucher au CLI ou au réseau.

En vrai

Extrais une fonction pure de ton atelier CSV. Écris 3 asserts. Fais-en échouer un volontairement pour voir le message, puis réparer. Note le temps : souvent moins de dix minutes. Ces dix minutes te sauveront des soirées.

A toi

Si tu es à l'aise, installe pytest et déplace tes asserts dans `test_*.py`. Sinon, garde un fichier `checks.py` lanceable. L'habitude compte plus que l'outil. Bonus : ajoute le cas "note non numérique" si tu le gères dans ta fonction. Lea lance ses checks avant chaque livraison client. Max aussi, depuis un devis faux. Sam exige `checks.py` sur les projets élèves partagés.

ATTENTION

Ne teste pas seulement le chemin heureux. Ajoute le cas vide, le cas absent, et un cas limite. C'est là que ça casse demain.

Chapitre 18 - Organiser un petit projet



src, tests, README, requirements, venv.

Un script unique, c'est bien pour apprendre. Des qu'il y a CSV + API + CLI + tests, un peu d'ordre evite le chaos. Pas besoin d'architecture d'entreprise. Juste un dossier clair. Un nouvel arrivant (souvent toi dans trois semaines) doit comprendre en trente secondes ou cliquer.

Trois zones : donnees (`data/`), code (`src/` ou fichiers a plat), tests (`tests/`). A cote : venv (ignore), requirements (partage), secrets (`.env` ignore, `.env.example` partage). Le point d'entree parse les arguments et appelle la logique. La logique ne connait pas argparse. Lea dessine parfois ces zones sur papier avant de deplacer quoi que ce soit. Max aussi, depuis qu'il a perdu un CSV dans un dossier "Nouveau dossier (3)".

Chez DanielCraft, on vise exactement ca. Lea range ses scripts clients dans une arborescence simple. Max a supprime ses `script_final_v2_REEL.py` et un seul dossier propre. Sam partage son projet notes avec un README de quinze lignes que tout le monde comprend. L'ordre n'est pas du perfectionnisme. C'est du temps gagne et de la peur en moins.

Ce que ce n'est pas

Organiser, ce n'est pas recopier l'arborescence d'un monorepo Google. Ce n'est pas non plus renommer vingt fois pour le plaisir. Ce n'est pas "src obligatoire des la ligne 1". Si deux fichiers suffisent, reste a plat. Et ce n'est surtout pas committer le `.venv` "pour etre sur" : c'est lourd, inutile, parfois fragile.

★ A RETENIR

Ecris la structure sur papier avant de deplacer les fichiers. Dix minutes d'ordre economisent une heure de chasse.

Arborescence type

```
mon-outil/  
  .venv/  
  .gitignore  
  .env.example  
  requirements.txt  
  README.md  
  data/  
    notes.csv  
  src/  
    mon_outil/  
      __init__.py  
      cli.py  
      notes.py  
      meteo.py  
  tests/  
    test_notes.py
```

Variante plus simple, tout a plat au debut :

```
mon-outil/  
  .venv/  
  requirements.txt  
  resume.py  
  notes_lib.py  
  data/  
  tests/
```

L'important : separer donnees, code, tests. Ne pas melanger le CSV de prod et les brouillons sans nom. Sam colle un fichier `notes.exemple.csv` a cote du vrai, pour que les eleves ne cassent pas les donnees reelles.

requirements.txt

```
requests==2.32.3  
pytest==8.3.0
```

Les versions exactes (pin) rendent les installs reproductibles. Pour un apprentissage, pinner au moins les paquets critiques.

Installation :

```
python -m venv .venv  
# activer le venv  
python -m pip install -r requirements.txt
```

README court

Quatre blocs suffisent : a quoi sert l'outil, comment installer, comment lancer (2 exemples), ou mettre sa cle si besoin (renvoi a `.env.example`). Ecris pour un humain presse, pas pour un roman. Lea lit ses README a voix haute : si elle hesite, elle raccourcit.

.gitignore utile

```
.venv/  
.env  
__pycache__/  
*.pyc  
.pytest_cache/  
app.log
```

Tu ignores le virtuel, les secrets, le cache Python, les logs locaux. Chez DanielCraft, ce fichier est non negociable des qu'il y a un depot.

Ou mettre le point d'entree ?

`cli.py` ou `resume.py` parse les arguments et appelle des fonctions. `notes.py` / `meteo.py` portent la logique. Les tests importent la logique, pas le parseur complet (sauf tests d'integration plus tard). Max a longtemps tout mis dans un seul fichier. Ca marchait. Puis il a voulu tester. La, ca a cesse de marcher confortablement. Decoupage.

Variante : tout a plat au debut

Si ton projet tient en deux fichiers, ne force pas une arborescence lourde. Lea commence souvent avec `resume.py` + `notes_lib.py` + `data/` + `tests/`. Quand un troisieme module apparait (`meteo`, `config`, `export`), elle decoupe. La regle : separe des que tu te perds, pas avant. Sam partage ses projets eleves avec cette structure minimale. Max a migre vers `src/` seulement quand il a eu cinq scripts qui se parlaient entre eux.

Commandes types dans le README

Un bon README inclut les commandes copiables :

```
python -m venv .venv  
.\.venv\Scripts\Activate.ps1  
python -m pip install -r requirements.txt  
python resume.py --mode notes --fichier data/notes.csv --eleve Alice  
python resume.py --mode meteo --lat 48.85 --lon 2.35  
pytest
```

Sam colle ce bloc en haut de ses projets eleves. Lea ajoute une ligne "Variables : voir `.env.example`". Max note quel venv activer selon le dossier. Dix lignes de README, zero reunion pour expliquer comment lancer.

Petite histoire

Max avait un dossier "Nouveau dossier (3)" avec quatre versions de son script `meteo`. Lea lui a fait passer une heure a ranger : un dossier, un README, un `requirements.txt`. Le lendemain, Max a retrouve son outil en dix secondes au lieu de dix minutes. L'ordre, ce n'est pas du perfectionnisme. C'est du temps gagne.

Autre scene : Sam recoit un projet eleve sans README. Il ouvre, ferme, renvoie : "quinze lignes, puis on regarde le code". L'eleve rale. Puis remercie. Chez DanielCraft, on valide ce rituel sans honte.

Erreur classique

Un dossier `Nouveau dossier (3)` avec `script_final_v2_REEL.py`. Ou installer sans `requirements.txt` et ne plus savoir quoi recréer sur une autre machine. Ou committer `.venv` (lourd, inutile, parfois fragile). Ou mélanger données réelles et brouillons de test sans distinction. Autre piège : six fichiers `utils.py` qui s'appellent tous pareil dans des sous-dossiers. Nomme clairement.

⚠ ATTENTION

Ne committe jamais `.env` ni `.venv`. Partage `.env.example` et `requirements.txt`. C'est le minimum vital.

En vrai

Reprends ton mini-projet. Range-le dans une arborescence simple. Ajoute `requirements.txt`, `.env.example`, un README de 15 lignes. Supprime les fichiers morts. Lance depuis le README sans tricher. Si tu bloques, le README est trop flou : corrige-le.

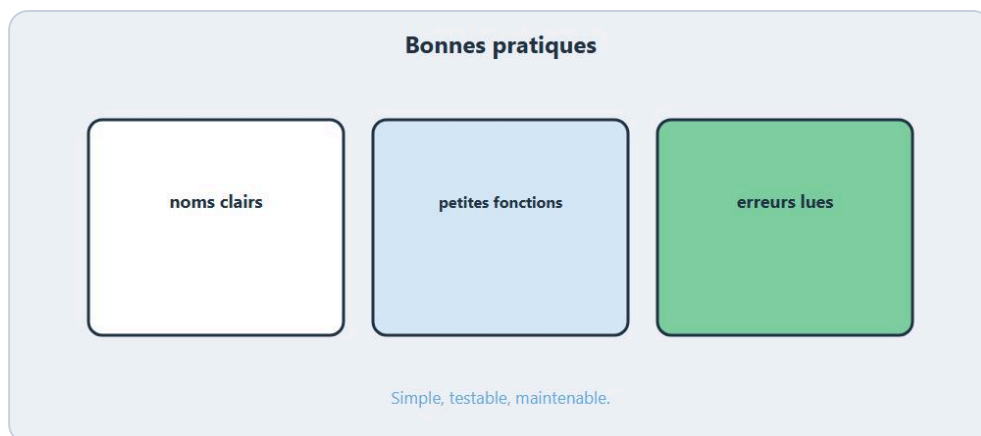
A toi

Ecris la structure choisie sur papier (ou dans le README) avant de déplacer les fichiers. Dix minutes d'ordre économisent une heure de "ou j'avais mis ça ?". Bonus : demande à quelqu'un de lancer ton outil uniquement avec le README. Note où il bute. Si la personne bloque, corrige le README avant de "expliquer à l'oral" : l'oral ne voyage pas avec le dépôt.

♦ ASTUCE

Si quelqu'un lance ton outil uniquement avec le README et bute, c'est le README qui est trop flou - pas ton collègue.

Chapitre 19 - Bonnes pratiques



Noms clairs, petites fonctions, erreurs lues.

Ce chapitre ne rajoute presque pas de syntaxe. Il solidifie des reflexes. Tu peux le relire de temps en temps, comme une checklist amicale. Chez DanielCraft, on prefere trois habitudes ancrées à dix intentions oubliées. Ce n'est pas un sermon. C'est une boîte à outils mentale pour le jour où tu livres, partages, ou reprends un script à froid.

Lea relit cette liste avant de livrer un script client. Max l'a collée en commentaire en haut de ses outils perso. Sam en parle en fin de trimestre : ce ne sont pas des règles scolaires, ce sont des gestes qui évitent la galère. Toi, tu en choisis deux cette semaine. Pas vingt. Deux.

★ A RETENIR

Noms clairs, échecs prévisibles, secrets dehors, venv isole. Quatre piliers. Le reste suit.

Fonctions courtes, noms clairs

`moyenne_eleve` dit ce qu'elle fait. `calc2` non. Une fonction qui lit un CSV, appelle le réseau, formate trois messages et écrit un fichier... c'est trop. Découpe. Chez DanielCraft, on préfère trois fonctions ennuyeuses à une fonction héros. Lea renomme avant de refactoriser. Max a appris après un devis faux. Sam barre les noms flous au tableau.

Un seul rôle pour le point d'entrée

`main` parse, appelle, affiche. La logique métier vit ailleurs. Bénéfice : tests plus simples, CLI plus mince. Tu as vu ça au chapitre tests. Ici, tu l'ancres comme habitude, pas comme théorie.

Échecs prévisibles

Fichier absent, JSON inattendu, réseau down : tu les as déjà vus. Traite-les tout. Messages en français simple. Détails techniques dans les logs. Code retour non zéro (`sys.exit(1)`) quand le script est utilisé dans une chaîne d'outils. L'humain lit une phrase. La machine lit un code. Les deux comptent.

UTF-8 et chemins

Toujours `encoding="utf-8"` pour le texte français. Préférer `pathlib`. Éviter les chemins absolus graves dans le code (`C:\Users\...`) : passe-les en argument ou relative au projet. Lea a déjà écrit un script qui marchait chez elle et cassait chez le client à cause d'un chemin absolu. Depuis, argument ou relatif. Point.

Dependances sous contrôle

Venv. `requirements.txt`. `python -m pip`. Pas de "j'ai installé globalement, tant pis". Moins de mystères entre machines. Sam refuse de déboguer un projet élevé sans venv activé. Max aussi, depuis qu'il a cassé un autre outil en mettant à jour un paquet global.

Secrets

Variables d'environnement. `.env` ignoré par Git. `.env.example` partageable. Jamais de jeton dans un screenshot de cours ni dans un log. Chez DanielCraft, c'est non négociable. Une clé fuitée, c'est une soirée gâchée et parfois une facture.

Peu de magie

Les regex, les one-liners cryptiques, les imports circulaires : souvent impressionnants, rarement gentils pour le futur toi. Clairette > densité. Si tu as besoin d'un commentaire pour expliquer une ligne, demande-toi si tu peux écrire trois lignes plus claires à la place.

Documenter le "pourquoi" utile

Un README court. Une docstring sur une fonction non évidente. Pas besoin de commenter `i += 1`. Commente l'intention quand le code seul ne suffit pas : "Open-Meteo renvoie la temp en C sous `current_weather`". Lea écrit le pourquoi. Max écrit l'exemple d'appel. Sam exige les deux sur les projets partagés.

Versionne tes données d'exemple

Un petit `data/notes.exemple.csv` aide. Évite de casser le CSV "reel" à chaque test. Copie, travaille, compare. Sam le fait systématiquement. Lea aussi, après avoir écrasé un export client une fois. Une fois suffit.

Petit, mais relancable

Un bon script se relance demain sans que tu te souviennes de tous les détails. L'aide `-h`, le README, et des exemples d'appel dans un commentaire en haut du fichier aident énormément. Écris pour ton futur toi fatigué. C'est la vraie audience.

Éviter la perfection prématurée

Pas besoin de microservices pour un CSV de notes. Pas besoin de dix fichiers pour trente lignes. Organise assez pour rester lisible. Si tu passes plus de temps à "architecturer" qu'à faire marcher le cas nominal, simplifie. Chez DanielCraft, on coupe les châteaux avant les fondations.

Code retour et scripts en chaine

Quand ton script est appele par un autre outil (cron, tache planifiee, Makefile), le code retour compte. `sys.exit(0)` si tout va bien, `sys.exit(1)` si echec. L'appelant peut reagir sans lire le texte affiche. Max a decouvert ca quand son script meteo "marchait" mais envoyait quand meme un mail d'alerte parce qu'il retournait 1 par erreur. Une ligne, un gros bug en moins.

Petite checklist avant de partager

Est-ce que `-h` est clair ? Est-ce qu'un fichier manquant est explique ? Est-ce que le venv est documente ? Est-ce qu'un secret traîne ? Est-ce qu'un test minimal existe pour le coeur du calcul ? Si oui, ton script merite d'etre montre. Sinon, corrige deux cases, puis montre. Pas besoin d'attendre la perfection.

Petite histoire

Lea a livre un script sans README ni requirements. Le client a mis trois jours a le faire tourner. Lea a ajoute quinze lignes de doc et un requirements.txt. Le client a relance en dix minutes. La lecon : un script pratique, c'est aussi un script qu'on peut reprendre sans toi.

Max a partage un outil avec une cle en dur "juste pour la demo". La demo a fuite dans un salon. Il a tourne la cle, appris la lecon, colle `.env.example` partout depuis. Sam raconte cette histoire (anonymisee) chaque annee. Elle marche mieux qu'un slide "securite".

Erreur classique

Vouloir appliquer les vingt points d'un coup et ne rien ancrer. Ou traiter ce chapitre comme de la litterature : lu, admire, oublie. Autre piege : perfectionner l'architecture pendant que le cas nominal ne marche pas encore. Fais marcher. Puis solidifie. Dans cet ordre.

⚠ ATTENTION

Une cle dans le code "juste pour la demo", c'est deja une fuite en puissance. `.env`, toujours.

En vrai

Relis ton mini-projet avec cette checklist. Corrige deux choses seulement, mais pour de vrai. Les bonnes pratiques arrivent par couches, pas par sermon. Demain, deux autres. Dans une semaine, ton script aura change de visage sans drame.

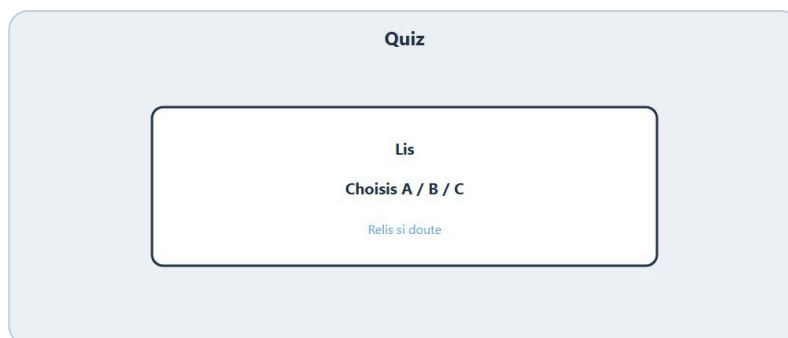
A toi

Choisis une habitude (timeout, venv, ou secrets) et applique-la systematiquement a tous tes scripts cette semaine. Une habitude ancree vaut mieux que dix intentions. Ecris l'habitude sur un post-it. Coches chaque script traite. Simple, visible, efficace. Si tu rates un jour, ne recommence pas a zero : reprends le script suivant. Chez DanielCraft, la constance bat la perfection.

♦ ASTUCE

Deux habitudes cette semaine, pas vingt. Une ancree bat dix intentions sur un post-it oublie.

Quiz final



Vérifier sans inventer.

Pas de piège. Relis une fois si besoin. L'idée : vérifier que tu peux avancer sans paniquer. Chez DanielCraft, on considère qu'un quiz réussi, c'est surtout la preuve que tu peux relancer un script demain sans relire vingt pages. Lea note ses trous. Max refait un atelier. Sam utilise ce QCM en fin de module. Toi, tu coches sans tricher. Ensuite seulement, tu compares.

Une première passe honnête vaut mieux qu'un score maquillé. Si une question te bloque, marque-la et continue. Tu reviendras. Le but n'est pas la perfection du jour. Le but, c'est la carte des chapitres à rouvrir.

★ A RETENIR

Fais le quiz, note ton score, rejoue dans une semaine. Le vrai progrès se voit à la deuxième passe.

Questions

1. 1. Pourquoi préférer `pathlib.Path` à une string de chemin collée à la main ?
 - A) Parce que Path colore le terminal
 - B) Pour manipuler des chemins proprement (joindre, vérifier, lire) sans galérer avec les slash
 - C) Parce que CSV l'exige
1. 2. Avec `csv.DictReader`, chaque ligne est surtout :
 - A) Un entier
 - B) Un dictionnaire dont les clés viennent de l'en-tête
 - C) Une image
1. 3. À quoi sert `argparse` ?
 - A) À installer des paquets
 - B) À définir et lire les arguments d'un script en ligne de commande
 - C) À créer des pages web
1. 4. Un environnement virtuel (`venv`), c'est surtout pour :
 - A) Isoler les paquets d'un projet
 - B) Remplacer Git
 - C) Compiler le système

1. 5. Quelle commande installe souvent les dependances listees ?

- A) `python -m pip install -r requirements.txt`
- B) `csv install requirements`
- C) `pathlib --install`

1. 6. Dans `requests`, que fait souvent `reponse.json()` ?

- A) Ouvre Excel
- B) Parse le corps JSON en structures Python (dict/list)
- C) Cree un venv

1. 7. Pourquoi mettre un `timeout` sur un GET ?

- A) Pour colorier les logs
- B) Pour eviter d'attendre indefiniment si le reseau/serveur rame
- C) Parce que CSV l'impose

1. 8. `raise_for_status()` sert surtout a :

- A) Ignorer les erreurs
- B) Lever une exception si le code HTTP indique une erreur (4xx/5xx)
- C) Formater une date

1. 9. Ou stocker une cle API plutot que dans le code source ?

- A) Dans une variable d'environnement (`.env` hors Git)
- B) Dans le nom du fichier CSV
- C) Dans un commentaire `print`

1. 10. Le niveau `logging.INFO` par rapport a `DEBUG` :

- A) Affiche en general moins de detail que DEBUG
- B) Remplace argparse
- C) Chiffre les secrets

1. 11. `strftime` sert a :

- A) Installer pytest
- B) Formater une date/heure en chaine selon un motif
- C) Lire un CSV

1. 12. Une bonne raison d'utiliser `re.search` :

- A) Remplacer pathlib pour lister des dossiers
- B) Chercher un motif dans un texte irregulier (ex: email approximatif)
- C) Activer le venv

Corriges

1-B, 2-B, 3-B, 4-A, 5-A, 6-B, 7-B, 8-B, 9-A, 10-A, 11-B, 12-B.

Si tu as 9/12 ou plus : tu es pret pour de vrais petits outils. Sinon, relis les chapitres lies (chemins, CSV, CLI, reseau, secrets), sans dramatiser. Un 7/12 honnête avec une carte de reprise bat un 12/12 maquille qu'on oublie demain.

⚠ ATTENTION

Ne triche pas sur le score pour te rassurer. Note les trous. Le vrai progres se voit a la deuxieme passe, pas au premier coches.

Petite histoire

Lea a rate la question timeout. Elle a rit, puis a ajoute `timeout=10` partout dans ses scripts clients le soir meme. Max a confondu `strftime` et `strptime` : il a rejoue l'atelier dates dix minutes. Sam affiche le score moyen sans nommer personne : "on revoit secrets et venv demain". Trois reactions, meme message : le quiz est une carte, pas un jugement. Chez DanielCraft, on aime ces cartes.

Questions bonus (auto-eval)

13. Cite trois gestes pour un script reseau robuste.
14. Difference entre `print` et `logging` dans un CLI.
15. Pourquoi `Path(__file__).parent` plutot qu'un chemin relatif au dossier courant. Reponses libres : compare a tes chapitres 2, 7, 9.

En vrai

Coche tes mauvaises reponses. Ouvre le chapitre lie. Refais un mini geste de cinq minutes : un GET avec timeout, un Path, un assert. Pas besoin de tout relire. Un trou a la fois.

A toi

Ecris tes trois questions ratees (ou les plus floues). A cote, une phrase "ce que je confonds". Dans sept jours, rejoue seulement ces trois. Le vrai score, c'est la deuxieme passe.

Pour aller plus loin sans te perdre

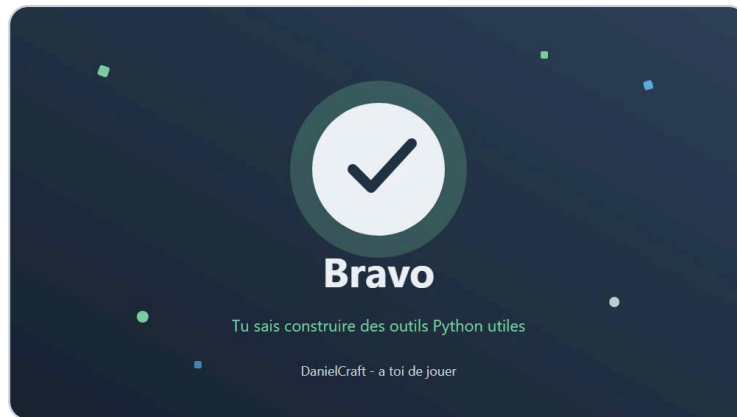
Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique du chapitre concerne. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre.

Si tu as rate timeout ou secrets, ne te flagelle pas. Rouvre le chapitre 7 ou 8 dix minutes, refais un geste concret, puis rejoue ces deux questions seules. Lea a fait exactement ca. Max aussi. Le score du jour compte moins que la reprise dans sept jours. Chez DanielCraft, on garde ce quiz comme carte de poche, pas comme jugement.

♦ ASTUCE

Trois relectures actives battent une lecture passive de vingt pages. Un trou a la fois, un geste de cinq minutes.

Chapitre 21 - Bravo



Bravo. Tu sais faire des scripts Python utiles.

Tu as parcouru un chemin solide : pathlib, CSV, argparse, venv et pip, requests et erreurs HTTP, secrets, logging, datetime, regex, mini-projet, trois ateliers, tests, organisation, bonnes pratiques, et ce quiz. Relis cette liste sans dramatiser. Ce n'est pas un catalogue a memoriser. C'est un paysage de gestes que tu sais rouvrir.

Ce n'est pas un diplôme magique. C'est mieux : une tête claire et des gestes. Tu sais lire un tableau CSV sans Excel a minuit. Tu sais lancer un script avec des options sans retoucher le code. Tu sais appeler une API avec timeout et messages clairs. Tu sais garder les secrets hors du source. Tu sais poser un test simple sur une fonction pure. Ces phrases, dites a voix haute, valent mieux qu'un certificat.

Tu as un petit atelier d'outils. Sur l'établi : chemins, CSV, CLI, venv, HTTP, secrets, logs, dates, tests. Tu sais ouvrir chaque tiroir. Tu ne connais pas encore toute l'usine d'à côté (web, bases, packaging). Ce n'est pas grave. Tu as les mains. Et les mains, en pratique, battent les catalogues. Le reste se construit sur ce socle, pas à côté.

Chez DanielCraft, on considère que c'est ça, la vraie compétence Python pratique en 2026. Pas connaître vingt bibliothèques. Automatiser son quotidien proprement. Lea repart avec des outils clients. Max repart avec météo et factures. Sam repart avec des ateliers rejouables. Toi, tu repars avec une habitude : petit outil, propre, relancable.

Ce que ce n'est pas

Ce bravo n'est pas "tu n'as plus rien à apprendre". Ce n'est pas non plus une invitation à tout jeter pour "refaire propre depuis zéro". Ce n'est pas le moment d'enchaîner trois frameworks. Améliore. Versionne. Continue. Le perfectionnisme du grand soir casse plus de progressions que les timeouts mal réglés.

★ A RETENIR

Petit outil, propre, relancable. Les mains battent les catalogues. Continue à servir ton quotidien.

Ce que tu sais faire maintenant

Manipuler des chemins avec `pathlib`. Lire et écrire du CSV proprement. Construire un CLI avec `argparse`. Créer un `venv` et installer avec `pip`. Faire un GET avec `requests`, avec timeout et vérification HTTP. Garder les secrets hors du code. Logger sans noyer l'utilisateur. Formater des dates. Utiliser une regex avec modération. Poser un test simple sur une fonction pure. Relis cette liste à voix haute. C'est ton paysage.

La suite

Reutilise tes scripts types. Refais le mini-projet sur une nouvelle tâche. Un vrai petit outil perso cette semaine : suivi de notes, export CSV, rappel météo du matin, nettoyage de fichiers. Plus tard, si tu veux : API web (Flask/FastAPI), automatisation plus poussée, bases de données, packaging. Mais tu as le socle pratique qui porte tout ça : fichiers, CLI, HTTP, hygiène de projet.

Un conseil pour la route

Garde un dépôt "outils" avec trois scripts qui te servent vraiment. Améliore-les quand ça frotte. Le muscle se construit en servant ton quotidien, pas en collectionnant des tutoriels. Lea garde ses trois scripts clients. Max a son météo et son CSV factures. Sam relance ses moyennes chaque trimestre. Trois usages, une même posture : outil petit, propre, réutilisable.

⚠ ATTENTION

Sans engagement de trente jours, le livre redevient un souvenir. Avec engagement, il devient un réflexe. Choisis une date, pas une intention vague.

Un dernier rappel

Python accélère. Toi, tu orientes. Garde le contrôle : `venv` isole, secrets dehors, messages clairs quand ça casse. Et quand un script te promet la magie sans gestion d'erreur, souviens-toi de ce livre : ça marche en démo, ça casse en vrai - si tu n'as pas préparé les échecs. Le chemin malheureux compte autant que le chemin heureux. Tu l'as vu assez de fois pour ne plus l'oublier.

Engagement de 30 jours

Chaque semaine : améliorer un script existant (README, test, timeout, `argparse`). J+30 : refaire le quiz et lancer ton mini-projet sur une nouvelle donnée. Tu transformeras ce livre en muscle. C'est le vrai bravo. Sans engagement, le livre redevient un souvenir. Avec engagement, il devient un réflexe.

Petite histoire

Lea a repris son premier script CSV trois mois plus tard. Elle a rit devant les chemins en dur. Puis elle a vu le chemin : `Path`, `argparse`, timeout, `.env`. Max a montré son météo à un collègue sans dire "c'est du Python". Le collègue a vu un outil clair. Sam a vu un élève lire un `HTTPError` sans paniquer. Chez DanielCraft, ces scènes valent mieux qu'un score. Elles disent : tu pilotes.

En vrai

Ouvre un de tes scripts aujourd'hui. Change une seule chose utile (timeout, README, `assert`). Lance. Range. C'est le premier jour de l'après-livre.

A toi

Ecris en trois lignes : ton prochain script, la donnée qu'il lit (fichier ou API), et le premier argument CLI que tu vas ajouter. Puis ouvre l'éditeur. Garde ce papier trente jours. Tu verras le chemin.

Bravo. Maintenant, va faire un usage utile, vérifie, et range le script. Si tu ne sais pas par où commencer demain matin, rouvre ton papier "A toi" et lance l'éditeur avant le café. Lea, Max, Sam : chacun a ses trois scripts. Toi aussi. C'est le vrai bravo, trente jours plus tard.

♦ ASTUCE

Trois scripts qui servent vraiment battent vingt tutoriels empilés. Améliore-les quand ça frotte.

Tu as les briques. Maintenant, construis.